

嵌入式系统

第三讲 ARM7指令系统

教师：郭玉臣

Mail:yuchenguo@tongji.edu.cn

前言

1.ARM程序的文件类型:

- ◆**C程序**: ARM开发中大部分程序使用C语言编写, 文件类型为“***.C**”;
- ◆**汇编程序**: 涉及到硬件底层操作的代码有时必须使用汇编语言编写, 文件类型为“***.S**”。

2.为什么学习ARM指令系统:

- ◆**操作系统移植**
- ◆**编写启动代码**
- ◆**方便程序调试**



本章学习重点

1. ARM处理器的寻址方式;
2. ARM指令的特点;
3. ARM指令的种类, 能完成哪些功能。



目录

- 1.ARM处理器寻址方式
- 2.指令集介绍
 - ARM指令集
 - Thumb指令集



目录

- 1.ARM处理器寻址方式
- 2.指令集介绍
 - ARM指令集
 - Thumb指令集



第3章 ARM7TDMI(-S)指令系统

■ 简介

ARM处理器是基于精简指令集计算机(RISC)原理设计的，指令集和相关译码机制较为简单。ARM7TDMI(-S)具有**32位ARM指令集**和**16位Thumb指令集**，ARM指令集效率高，但是代码密度低；而Thumb指令集具有较高的代码密度，却仍然保持ARM的大多数性能上的优势，它是ARM指令集的子集。所有的ARM指令都是可以有条件执行的，而Thumb指令仅有一条指令具备条件执行功能。ARM程序和Thumb程序可相互调用，相互之间的状态切换开销几乎为零。

第3章 ARM7TDMI(-S)指令系统

■ ARM指令集与Thumb指令集的关系



Thumb指令集
具有灵活、小
巧的特点

ARM指令集支
持ARM核所有
的特性，具有高
效、快速的特点

- ARM体系结构支持16位的Thumb指令集
 - Thumb指令集是ARM指令集的一个子集，允许指令编码为16位的长度
 - 指令对应
 - 所有的Thumb指令都有对应的ARM指令
 - 编程模型对应
 - Thumb的编程模型对应于ARM的编程模型
 - 子程序互相调用
 - 在应用程序的编写过程中，只要遵循一定调用的规则，Thumb子程序和ARM子程序就可以互相调用

■ Thumb指令的特性

- Thumb指令集中的数据处理指令的**操作数仍然是32位，指令地址也为32位**
- 大多数的Thumb指令是**无条件执行的**，而几乎所有的ARM指令都是**有条件执行的**
- 大多数的Thumb数据处理指令的**目的寄存器与其中一个源寄存器相同**
- Thumb指令的长度为**16位**，只用ARM指令一半的位数来实现同样的功能
 - 要实现特定的程序功能，所需的Thumb指令的条数较ARM指令多

- 空间效率、时间效率和功耗分析
 - 存储空间
 - Thumb代码约为ARM代码的60%~70%
 - Thumb代码使用指令数比ARM代码多约30%~40%
 - 访存速度
 - 使用32位存储器，ARM代码比Thumb代码快约40%
 - 使用16位存储器，Thumb代码比ARM代码快约40%~50%
 - 功耗分析
 - 使用Thumb代码，存储器功耗会降低约30%

- ARM指令集和Thumb指令集各有其优点
 - 对系统的性能有较高要求，应使用32位的存储系统和ARM指令集
 - 对系统的成本及功耗有较高要求，则应使用16位的存储系统和Thumb指令集
 - 若两者结合使用，充分发挥其各自的优点，会取得更好的效果

3.1 ARM处理器寻址方式

■ 寻址方式分类

寻址方式是根据指令中给出的地址码字段来实现寻找真实操作数地址的方式。ARM处理器具有8种基本寻址方式。

1. 寄存器寻址；

2. 立即寻址；

3. 寄存器移位寻址；

4. 寄存器间接寻址；

5. 基址寻址；

6. 多寄存器寻址；

7. 堆栈寻址；

8. 相对寻址。

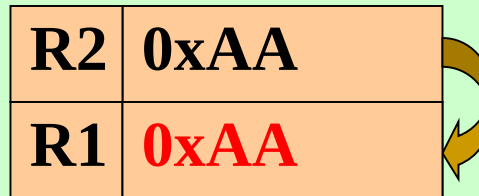
3.1 ARM处理器寻址方式

■ 寻址方式分类——寄存器寻址

操作数的值在寄存器中，指令中的地址码字段指出的是寄存器编号，指令执行时直接取出寄存器值来操作

MOV R1, R2

SUB R0, R1, R2



MOV R1,R2

3.1 ARM处理器寻址方式

■ 寻址方式分类——立即寻址

立即寻址指令中的操作码字段后面的地址码部分即是操作数本身，也就是说，数据就包含在指令中。

(这

SUB

MOV

程序存储

MOV R0, #0xFF00

R0 0xFF00

从代码中获得数据

MOV R0, #0xFF00

3.1 ARM处理器寻址方式

■ 寻址方式分类——寄存器移位寻址

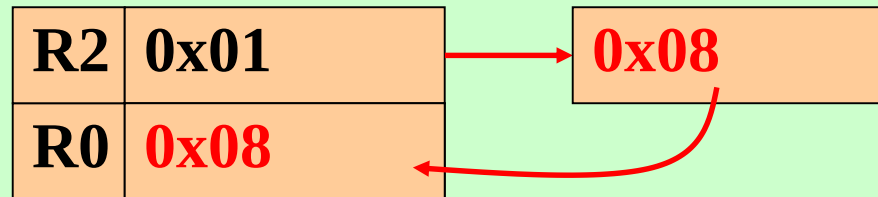
寄存器移位寻址是ARM指令集特有的寻址方式。
当第2个操作数是寄存器移位方式时，第2个寄存器

操
操
操

MOV

AND

逻辑左移3位



MOV R0,R2,LSL #3

3.1 ARM处理器寻址方式

■ 寻址方式分类——寄存器间接寻址

寄存器间接寻址指令中的地址码给出的是一个通用寄存器的编号，所需的操作数保存在寄存器指向的内存地址中。

定
针

LDR

SWP

0x40000000	0xAA
------------	------

R2	0x40000000
R0	0xAA

LDR R0,[R2]

3.1 ARM处理器寻址方式

■ 寻址方式分类——基址寻址

基址寻址就是将基址寄存器的内容与指令中给出的偏移量相加，形成操作数的有效地址。基址寻址

地址组
例：

LDR

STR

将R3+0x0C作为地址装载数据

0x4000000C	0xAA
------------	------

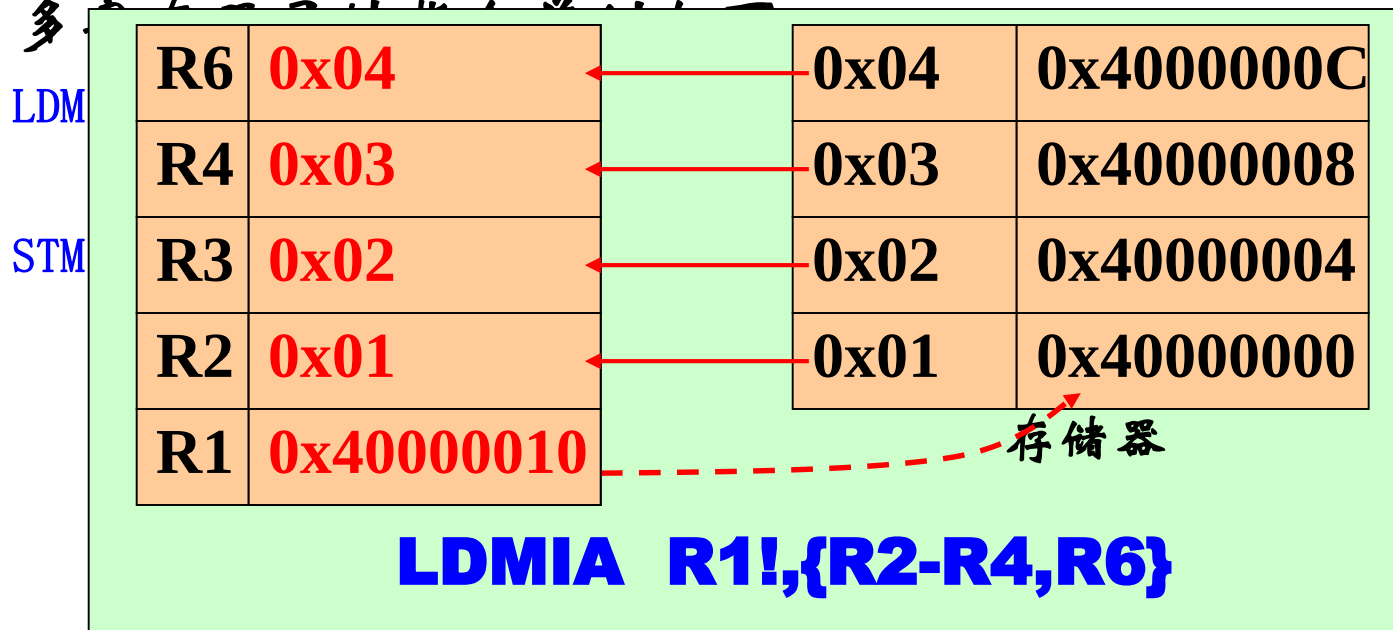
R3	0x40000000
R2	0xAA

LDR R2,[R3,#0x0C]

3.1 ARM处理器寻址方式

■ 寻址方式分类——多寄存器寻址

多寄存器寻址一次可传送几个寄存器值，允许一条指令传送16个寄存器的任何子集或所有寄存器。



3.1 ARM处理器寻址方式

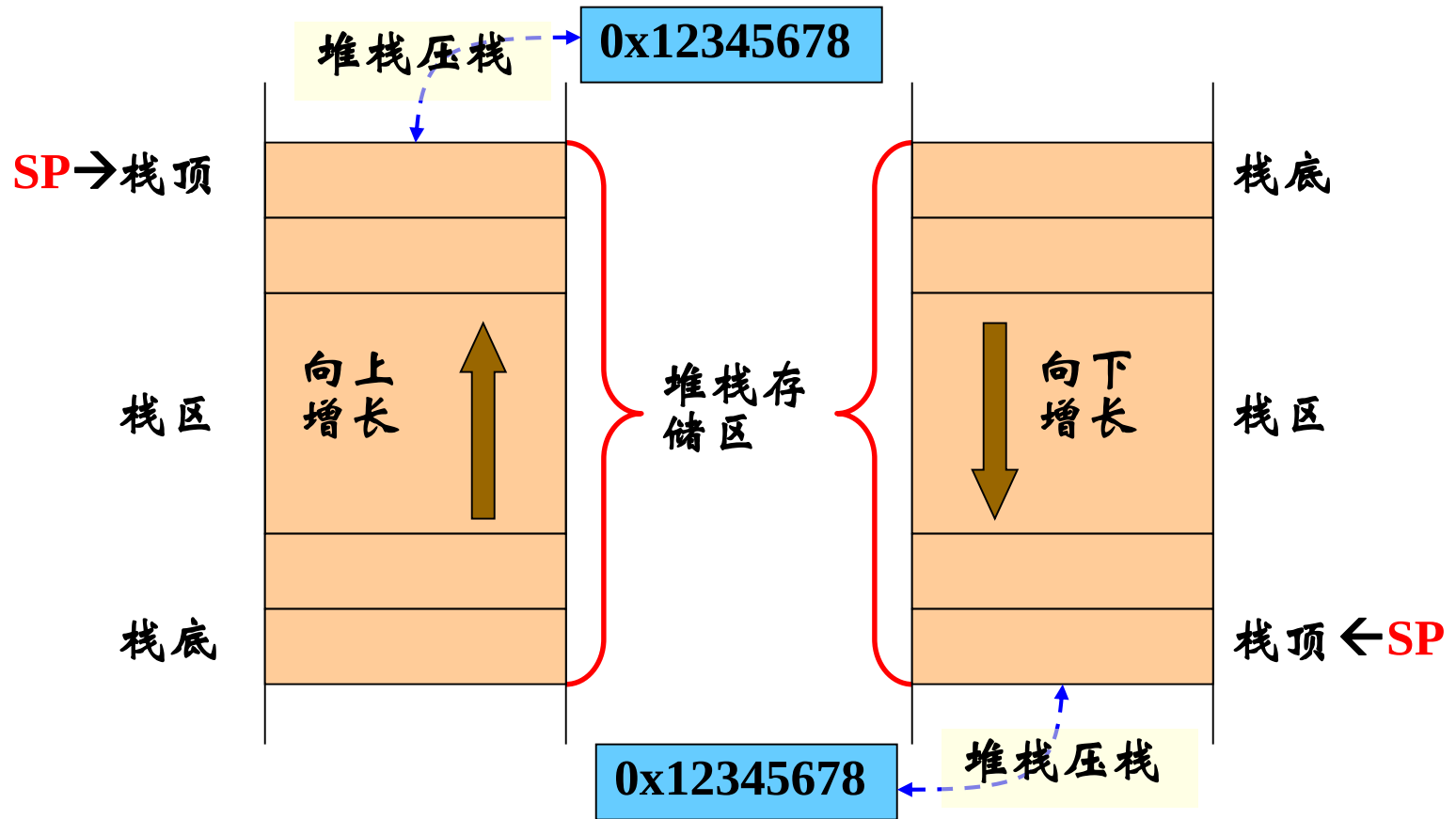
■ 寻址方式分类——堆栈寻址

堆栈是一个按特定顺序进行存取的存储区，操作顺序为“后进先出”。堆栈寻址是隐含的，它使用一个专门的寄存器(堆栈指针)指向一块存储区域(堆栈)，指针所指向的存储单元即是堆栈的栈顶。存储器堆栈可分为两种：

- 向上生长：向高地址方向生长，称为递增堆栈
- 向下生长：向低地址方向生长，称为递减堆栈

3.1 ARM处理器寻址方式

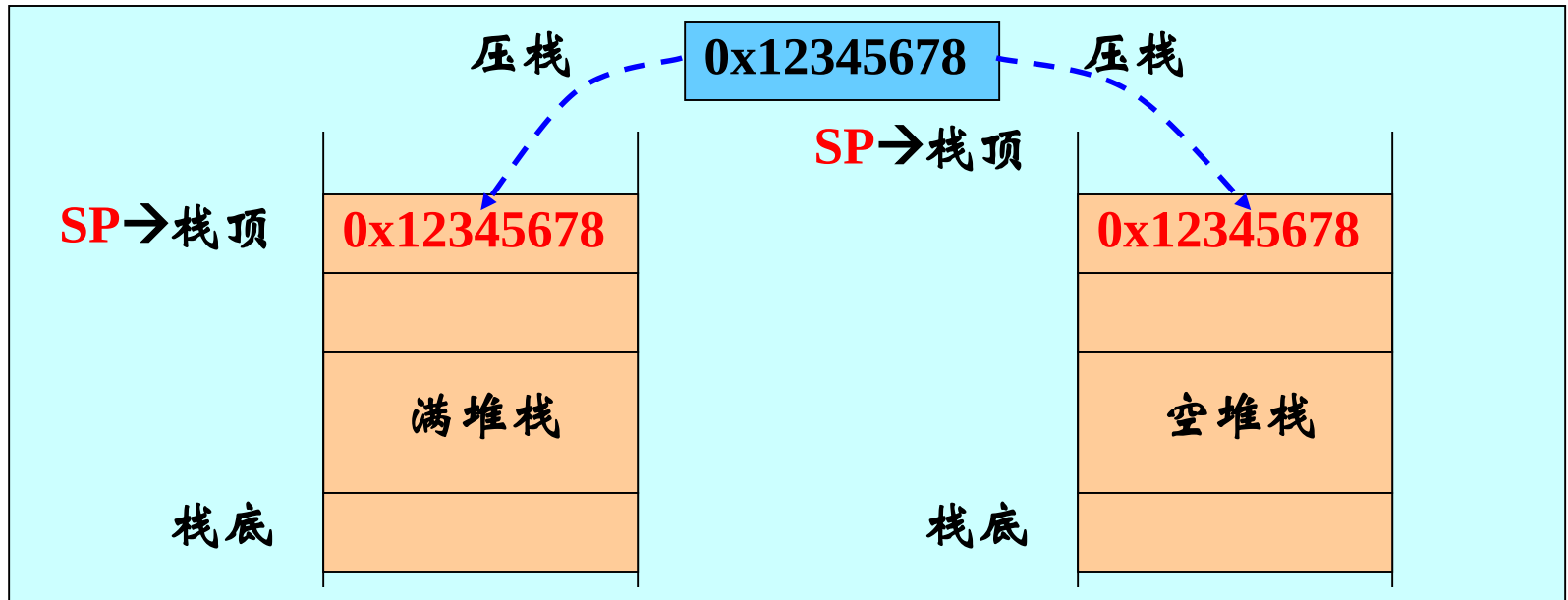
■ 寻址方式分类——堆栈寻址



3.1 ARM处理器寻址方式

■ 寻址方式分类——堆栈寻址

堆栈指针指向最后压入的堆栈的有效数据项，称为**满堆栈**；
堆栈指针指向下一个待压入数据的空位置，称为**空堆栈**。



3.1 ARM处理器寻址方式

■ 寻址方式分类——堆栈寻址

所以可以组合出四种类型的堆栈方式：

- **满递增**：堆栈向上增长，堆栈指针指向内含有效数据项的最高地址。指令如LDMFA、STMFA等；
- **空递增**：堆栈向上增长，堆栈指针指向堆栈上的第一个空位置。指令如LDMEA、STMEA等；
- **满递减**：堆栈向下增长，堆栈指针指向内含有效数据项的最低地址。指令如LDMFD、STMFD等；
- **空递减**：堆栈向下增长，堆栈指针向堆栈下的第一个空位置。指令如LDMED、STMED等。



3.1 ARM处理器寻址方式

■ 寻址方式分类——相对寻址

相对寻址是基址寻址的一种变通。由程序计数器PC提供基准地址，指令中的地址码字段作为偏移量，两者相加后得到的地址即为操作数的有效地址。相对寻址指令举例如下：

	BL	SUBR1	;调用到SUBR1子程序
	BEQ	LOOP	;条件跳转到LOOP标号处
	...		
LOOP	MOV	R6, #1	
	...		
SUBR1	...		

目录

- 1.ARM处理器寻址方式
- 2.指令集介绍
 - ARM指令集
 - Thumb指令集

■ 简单的ARM程序

```

;文件名: TEST1.S
;功能: 实现两个寄存器相加
;说明: 使用“;”进行注释调试

        AREA      Example1, CODE, READONLY      ;声明代码段Example1
        ENTRY                                           ;标识程序入口
        CODE32                                           ;声明32位ARM指令
        START
        MOV        R0, #0                               ;设置参数
        MOV        R1, #10
        LOOP
        BL         ADD_SUB                               ;调用子程序ADD_SUB
        E
        ADDS       R0, R0, R1                           ;R0 = R0 + R1
        MOV        PC, LR                               ;子程序返回
        END
    
```

标号顶格写

实际代码段

声明文件结束



■ 简单的ARM程序

;文件名: TEST1.S

;功能: 实现两个寄存器相加

;说明: 使用ARMulate软件仿真调试

```
        AREA      Example1, CODE, READONLY      ;声明代码段Example1
        ENTRY                                           ;标识程序入口
        CODE32                                           ;声明32位ARM指令
START    MOV      R0, #0                                ;设置参数
        MOV      R1, #10
LOOP     BL       ADD_SUB                               ;调用子程序ADD_SUB
        B        LOOP                                  ;跳转到LOOP
ADD_SUB
        ADDS     R0, R0, R1                            ;R0 = R0 + R1
        MOV      PC, LR                                ;子程序返回
        END                                           ;文件结束
```

目录

- 1.ARM处理器寻址方式
- 2.指令集介绍
 - **ARM指令集**
 - **Thumb指令集**



ARM指令小節目录

1. 指令格式
2. 条件码
3. ARM指令

3.2 指令集介绍

■ ARM指令集——指令格式

ARM指令的基本格式如下：

`<opcode> {<cond>} {S}      <Rd> , <Rn>{, <operand2>}`

其中`<>`号内的项是必须的，`{ }`号内的项是可选的。
各项的说明如下：

opcode：指令助记符； **cond**：执行条件；

S：是否影响CPSR寄存器的值；

Rd：目标寄存器； **Rn**：第1个操作数的寄存器；

operand2：第2个操作数；

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Cond	0	0	1	Opcode				S	Rn				Rd				Operand2											
Cond	0	0	0	0	0	0	A	S	Rd				Rn				Rs		1	0	0	1	Rm					
Cond	0	0	0	0	1	U	A	S	RdHi				RdLo				Rn		1	0	0	1	Rm					
Cond	0	0	0	1	0	B	0	0	Rn				Rd				0	0	0	0	1	0	0	1	Rm			
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn				

Data/Processing/
PSR Transfer

Multiply

Multiply Long

Single Data Swap

Branch and Exchange

实际指令语法格式为：

ADDEQS R0,R1,R2;

该指令的编码格式为：

31~28	27~25	24~21	20	19~16	15~12	11~0
<u>cond</u>		<u>opcode</u>	S	<u>Rn</u>	Rd	op2
0000	001	0100	1	0001	0000	000000000010

transfer:

transfer:

at

transfer

transfer

ta Transfer

ta Operation

Cond	1	1	1	0	CP Opc	L	CRn	Rd	CP#	CP	1	CRm
Cond	1	1	1	1	Ignored by processor							

Coprocessor Register Transfer

Software Interrupt

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

3.2 指令集介绍

■ ARM指令集——第2个操作数

ARM指令的基本格式如下：

`<opcode> {<cond>} {S}      <Rd> , <Rn> {, <operand2>}`

灵活的使用第2个操作数“**operand2**”能够提高代码效率。它有如下的形式：

- #immed_8r——常数表达式；
- Rm——寄存器方式；
- Rm,shift——寄存器移位方式；

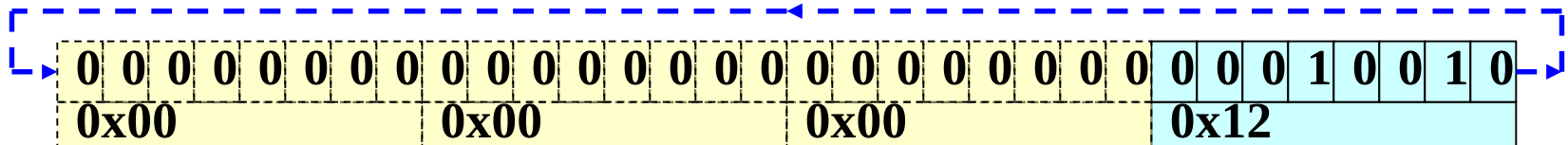
3.2 指令集介绍

■ ARM指令集——第2个操作数

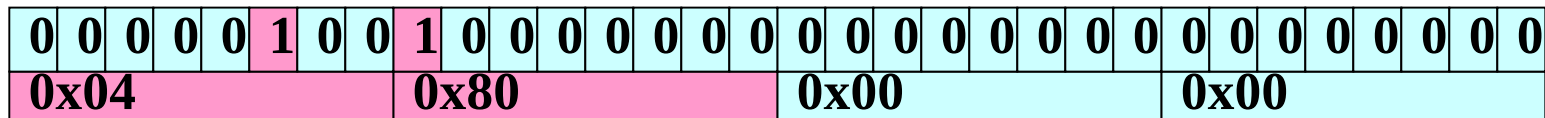
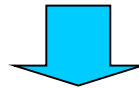
- #immed_8r——常数表达式

该常数必须对应8位位图，即必须是一个8位的常数通过循环右移偶数位可以得到的数。

循环右移10位



移位前的8位常数0x12



移位后得到的常数0x04800000

3.2 指令集介绍

■ ARM指令集——第2个操作数

■ #immed_8r——常数表达式

该常数必须对应8位位图，即必须是一个8位的常数通过循环右移偶数位可以得到的数。

例如：

MOV R0, #1

AND R1, R2, #0x0F

MOV R1, #0xC000 ;0xC000可由0x03循环右移16位得到



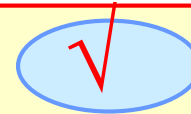
思考与练习

1. 以下8位图立即数是否合法？

0x0103C000



0x12800000



可以由0x4A循环右移10位得到

2. 请列举2个8位图立即数？

0x4000003B (0xED循环右移2位)

0x0016C000 (0x5B循环右移18位)

3.2 指令集介绍

■ ARM指令集——第2个操作数

■ Rm——寄存器方式

在寄存器方式下，操作数即为寄存器的数值。

例如：

SUB R1, R1, R2

MOV PC, R0

3.2 指令集介绍

■ ARM指令集——第2个操作数

■ Rm,shift——寄存器移位方式

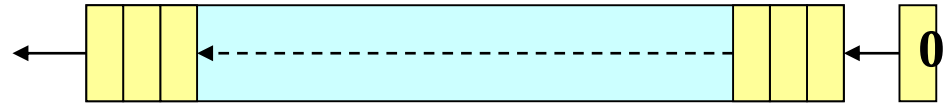
将寄存器的移位结果作为操作数，但Rm值保持不变，移位方法如下：

操作码	说明	操作码	说明
ASR #n	算术右移n位	ROR #n	循环右移n位
LSL #n	逻辑左移n位	RRX	带扩展的循环右移1位
LSR #n	逻辑右移n位	Type Rs	Type为移位的一种类型，Rs为偏移量寄存器，低8位有效。

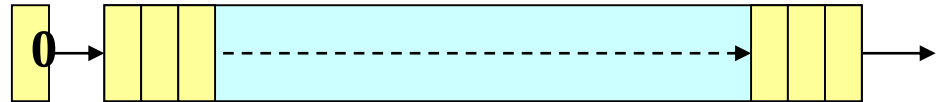
3.2 指令集介绍

■ ARM指令集——第2个操作数

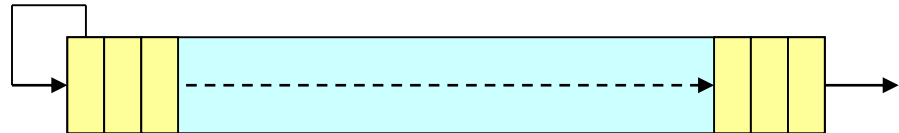
LSL移位操作:



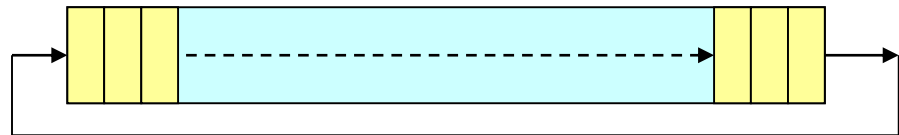
LSR移位操作:



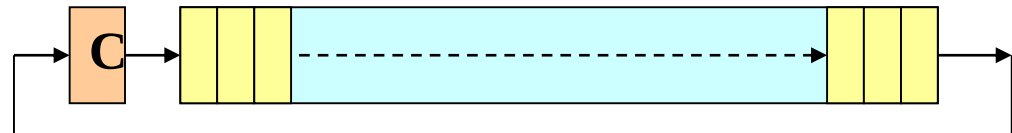
ASR移位操作:



ROR移位操作:



RRX移位操作:





3.2 指令集介绍

■ ARM指令集——第2个操作数

■ Rm,shift——寄存器移位方式

例如：

ADD R1, R1, R1, LSL #3 ;R1=R1+R1*8=9R1

SUB R1, R1, R2, LSR R3 ;R1=R1-(R2/2^{R3})



ARM指令小節目录

1. 指令格式
2. 条件码
3. ARM指令

3.2 指令集介绍

■ ARM指令集——条件码

ARM指令的基本格式如下：

<code><opcode> {<cond>} {S} <Rd> , <Rn> {, <operand2>}</code>

使用条件码“**cond**”可以实现高效的逻辑操作，提高代码效率。

绝大部分的ARM指令都可以条件执行，而Thumb指令只有B（跳转）指令具有条件执行功能。如果指令不标明条件代码，将默认为无条件（AL）执行。

指令条件码表

操作码	条件助记符	标志	含义
0000	EQ	Z=1	相等
0001	NE	Z=0	不相等
0010	CS/HS	C=1	无符号数大于或等于
0011	CC/LO	C=0	无符号数小于
0100	MI	N=1	负数
0101	PL	N=0	正数或零
0110	VS	V=1	溢出
0111	VC	V=0	没有溢出
1000	HI	C=1,Z=0	无符号数大于
1001	LS	C=0,Z=1	无符号数小于或等于
1010	GE	N=V	有符号数大于或等于
1011	LT	N!=V	有符号数小于
1100	GT	Z=0,N=V	有符号数大于
1101	LE	Z=1,N!=V	有符号数小于或等于
1110	AL	任何	无条件执行 (指令默认条件)
1111	NV	任何	从不执行(不要使用)

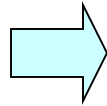
3.2 指令集介绍

■ ARM指令集——条件码

示例：

C代码：

```
if(a > b)
    a++;
else
    b++;
```



对应的汇编代码：

```
CMP      R0, R1      ;R0与R1比较
ADDHI    R0, R0, #1   ;若R0>R1, 则R0=R0+1
ADDLS    R1, R1, #1   ;若R0≤R1, 则R1=R1+1
```



本章学习重点

1. ARM处理器的寻址方式

——8种寻址方式；

2. ARM指令的特点

——可条件执行、可选择影响标志位、具有非常灵活的第二操作数；

3. ARM指令的种类，它能完成哪些功能。

ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令

为什么要掌握部分常用ARM指令？

- ◆ **熟悉ARM体系结构：**通过指令的学习可以更深入的了解ARM硬件结构的特点；
- ◆ **修改启动代码：**启动代码为了满足大部分系统的顺利运行，通常将系统硬件配置在最低性能，通过调整启动代码中的参数使其更适合自己的硬件系统；
- ◆ **调试程序：**通过观察反汇编代码了解程序执行情况，比如某个变量的操作是否被编译器优化掉了。
- ◆ **阅读已有的汇编代码；**

ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令

3.2 指令集介绍

■ ARM指令集——存储器访问指令

ARM处理器是典型的RISC处理器，对存储器的访问只能使用加载和存储指令实现。ARM处理器是冯·诺依曼存储结构，程序空间、RAM空间及I/O映射空间统一编址，除对RAM操作以外，对外围IO、程序数据的访问均要通过加载/存储指令进行。

存储器访问指令分为单寄存器操作指令和多寄存器操作指令。

3.2 指令集介绍

■ ARM存储器访问指令——单寄存器存取

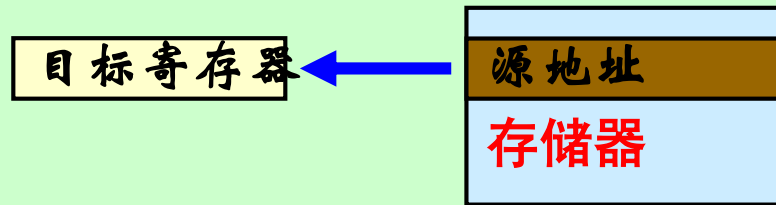
LDR/STR指令用于对内存变量的访问、内存缓冲区数据的访问、查表、外围部件的控制操作等。若使用LDR指令加载数据到PC寄存器，则实现程序跳转功能，这样也就实现了程序散转。

所有单寄存器加载/存储指令可分为“字和无符号字节加载存储指令”和“半字和有符号字节加载存储指令”。

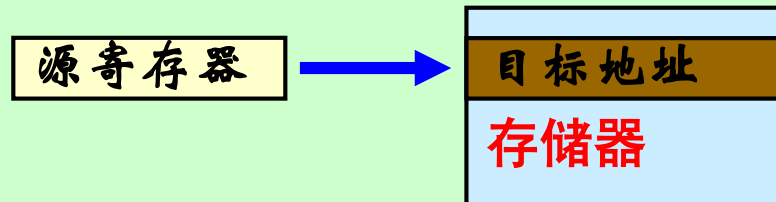
3.2 指令集介绍

■ ARM存储器访问指令——单寄存器存取

装载指令: **LDR** 目标寄存器,源地址



存储指令: **STR** 源寄存器,目标地址



3.2 指令集介绍

■ ARM存储器访问指令——单寄存器存取

装载指令: **LDR_x**

LDR/STR指令搭配不同的后缀实现不同方式的单寄存器存取操作:

存储指令: **STR_x**

字/半字/字节数据控制

是/否用户模式控制

无/有符号控制

ARM存储器访问指令——装载指令

助记符	说明	操作	条件码位置
LDR Rd, addressing	加载 字 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond}
LDR B Rd, addressing	加载无符号 字节 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} B
LDRT Rd, addressing	以 用户模式 加载 字 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} T
LDR BT Rd, addressing	以 用户模式 加载 无符号 字节数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} BT
LDR H Rd, addressing	加载 无符号半字 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} H
LDR SB Rd, addressing	加载 有符号字节 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} SB
LDR SH Rd, addressing	加载 有符号半字 数据	$Rd \leftarrow [addressing],$ addressing索引	LDR {cond} SH

• ARM存储器访问指令——保存指令

助记符	说明	操作	条件码位置
STR Rd, addressing	存储 字 数据	[addressing] ← Rd, addressing索引	STR{cond}
STR B Rd, addressing	存储 字节 数据	[addressing] ← Rd, addressing索引	STR{cond}B
STR T Rd, addressing	以 用户模式 存储 字 数据	[addressing] ← Rd, addressing索引	STR{cond}T
STR BT Rd, addressing	以 用户模式 存储 字节 数据	[addressing] ← Rd, addressing索引	STR{cond}BT
STR H Rd, addressing	存储 半字 数据	[addressing] ← Rd, addressing索引	STR{cond}H

ARM存储器访问指令——地址形式

装载指令: **LDR** 目标寄存器, **源地址**

保存指令: **STR** 源寄存器, **目标地址**

立即数: 立即数可以是一个无符号的数值。这个数据可以加到基址寄存器, 也可以从基址寄存器中减去这个数值。

如: LDR R1,[R0,#0x12]

寄存器: 寄存器中的数值可以加到基址寄存器, 也可以从基址寄存器中减去这个数值。

如: LDR R1,[R0,R2]

寄存器及移位常数: 寄存器移位后的值可以加到基址寄存器, 也可以从基址寄存器中减去这个数值。

如: LDR R1,[R0,R2,LSL #2]

ARM存储器访问指令——寻址方式

装载指令: **LDR** 目标寄存器, **源地址**

保存指令: **STR** 源寄存器, **目标地址**

● 零偏移: 如: LDR Rd,[**Rn**]

● 前索引偏移: 如: LDR Rd,[Rn,**#0x04**]!

● 程序相对偏移: 如: LDR Rd,**label1**

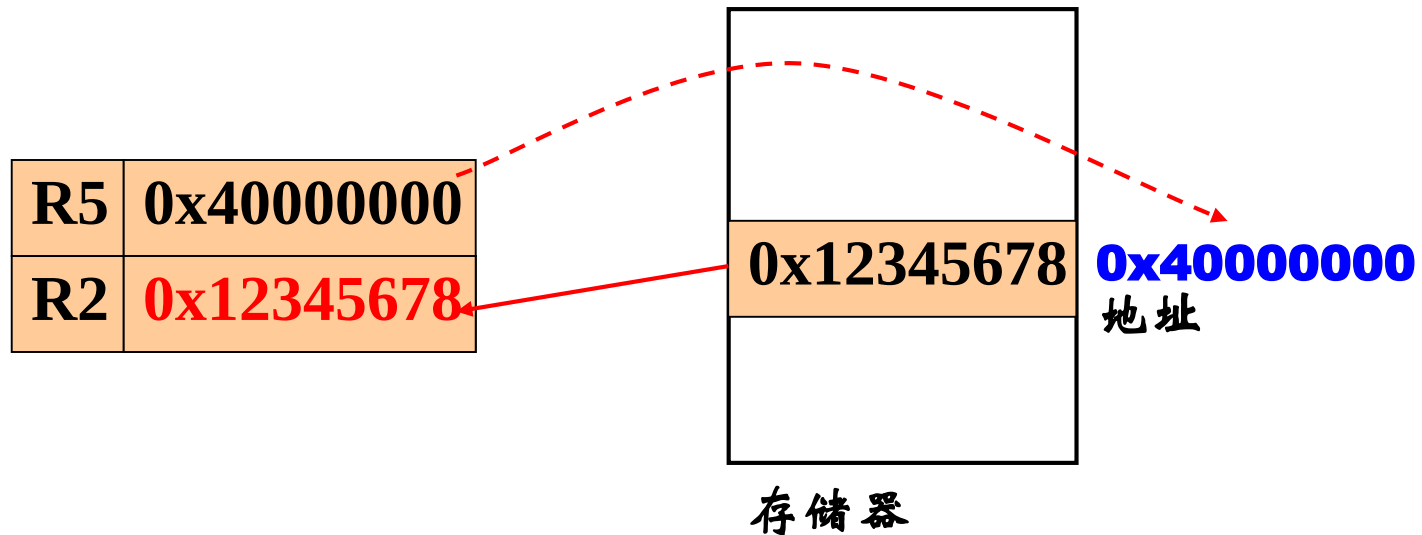
● 后索引偏移: 如: LDR Rd,[Rn],**#0x04**

注意: 大多数情况下, 必须保证字数据操作的地址是32位对齐的。

ARM存储器访问指令——单寄存器转载应用

应用示例：

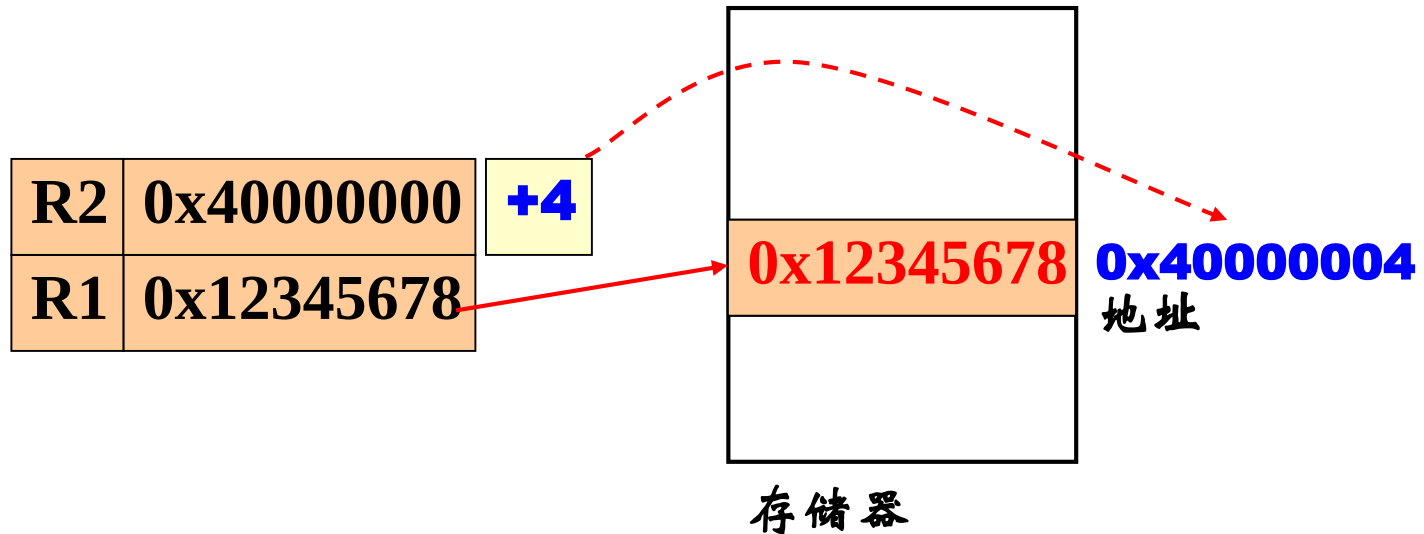
LDR R2,[R5] ;将R5指向地址的字数据存入R2



• ARM存储器访问指令——单寄存器保存应用

应用示例：

STR R1,[R2,#0x04] ;将R1的数据存储到R0+0x04地址



3.2 指令集介绍

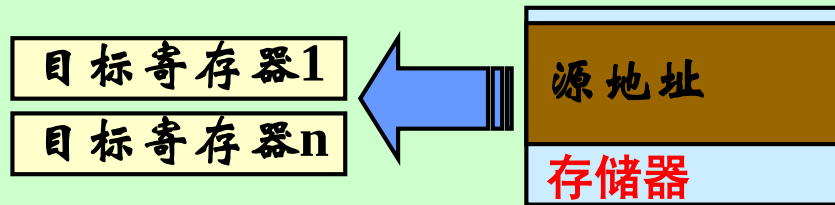
■ ARM存储器访问指令——多寄存器存取

多寄存器加载/存储指令可以实现在一组寄存器和一块连续的内存单元之间传输数据。LDM为加载多个寄存器；STM为存储多个寄存器。允许一条指令传送16个寄存器的任何子集或所有寄存器。它们主要用于现场保护、数据复制、常数传递等。

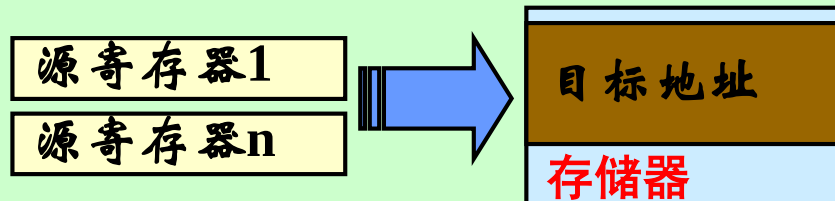
3.2 指令集介绍

■ ARM存储器访问指令——多寄存器存取

装载指令: **LDM** 源地址,目标寄存器列表



存储指令: **STM** 目标地址,源寄存器列表



3.2 指令集介绍

■ ARM存储器访问指令——多寄存器存取

装载指令: **LDM_x**

LDM/STM指令搭配不同的后缀实现不同方式地址增长方式:

存储指令: **STM_x**

IA: 每次传送后地址加4

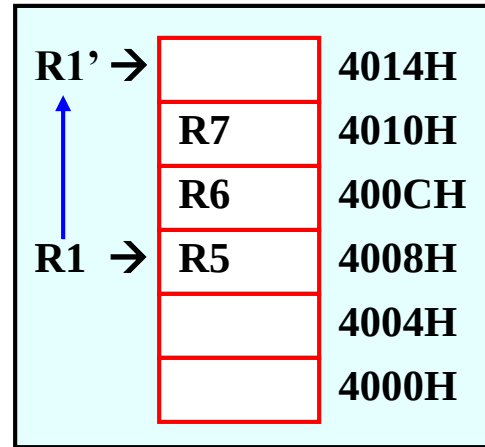
IB: 每次传送前地址加4

DA: 每次传送后地址减4

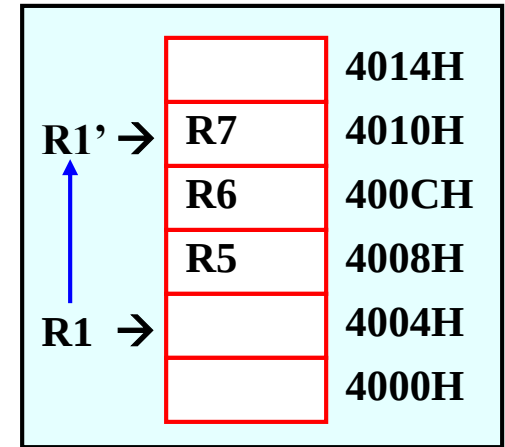
DB: 每次传送前地址减4

ARM存储器访问指令——多寄存器存取

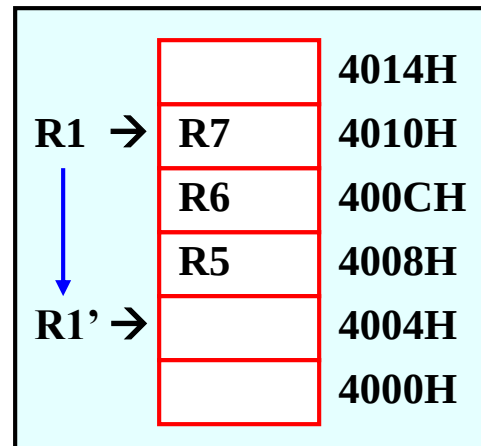
数据块传送指令操作过程如右图所示，其中R1为指令执行前的基址寄存器，R1'则为指令执行后的基址寄存器。



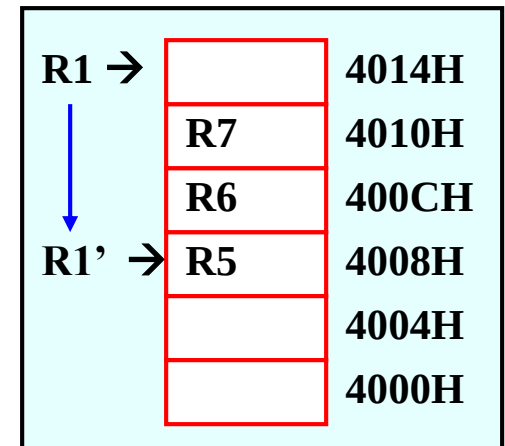
指令STM**IA** R1!,{R5-R7}



指令STM**IB** R1!,{R5-R7}



指令STM**DA** R1!,{R5-R7}



指令STM**DB** R1!,{R5-R7}

3.2 指令集介绍

■ ARM存储器访问指令——多寄存器存取

多寄存器存取指令与堆栈操作指令的关系如下表所示。

模式	说明	模式	说明
IA	每次传送后地址加4	FD	满递减堆栈
IB	每次传送前地址加4	ED	空递减堆栈
DA	每次传送后地址减4	FA	满递增堆栈
DB	每次传送前地址减4	EA	空递增堆栈
数据块传送操作		堆栈操作	

3.2 指令集介绍

■ ARM存储器访问指令——多寄存器存取

多寄存器存取指令与堆栈操作指令的关系如下表所示。

数据块传送 存储	堆栈操作 压栈	说明
STMD A	STMED	空递减
STM I A	STM E A	空递增
STM D B	STM F D	满递减
STM I B	STM F A	满递增

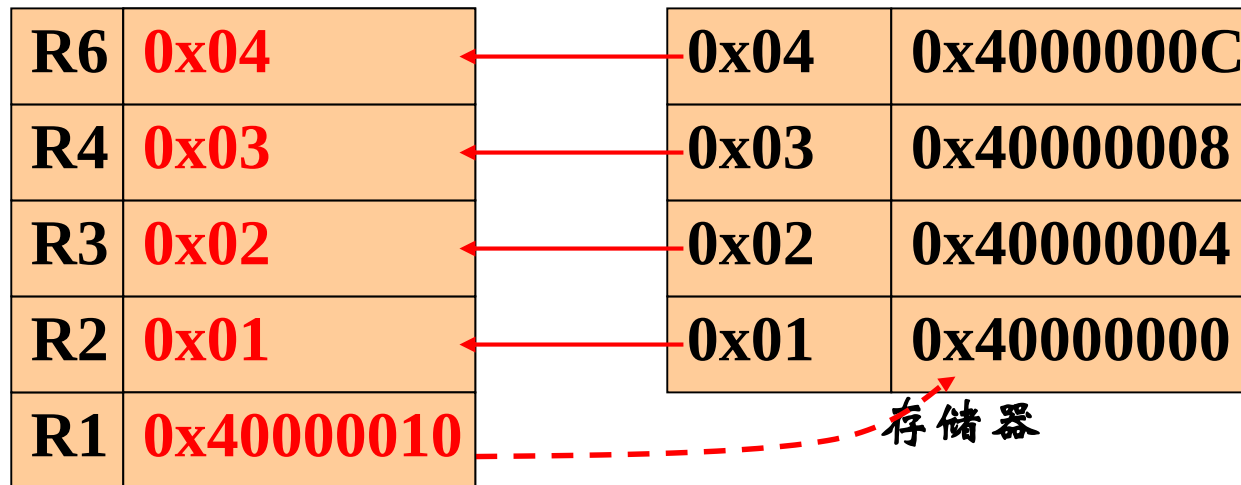
数据块传送 加载	堆栈操作 出栈	说明
LDM D A	LDM F A	满递减
LDM I A	LDM F D	满递增
LDM D B	LDM E A	空递减
LDM I B	LDM E D	空递增

ARM存储器访问指令——多寄存器存取

应用示例：

LDMIA R1!,{R2-R4,R6}

将R1指向的内存数据读取到R0-R4和R6寄存器中



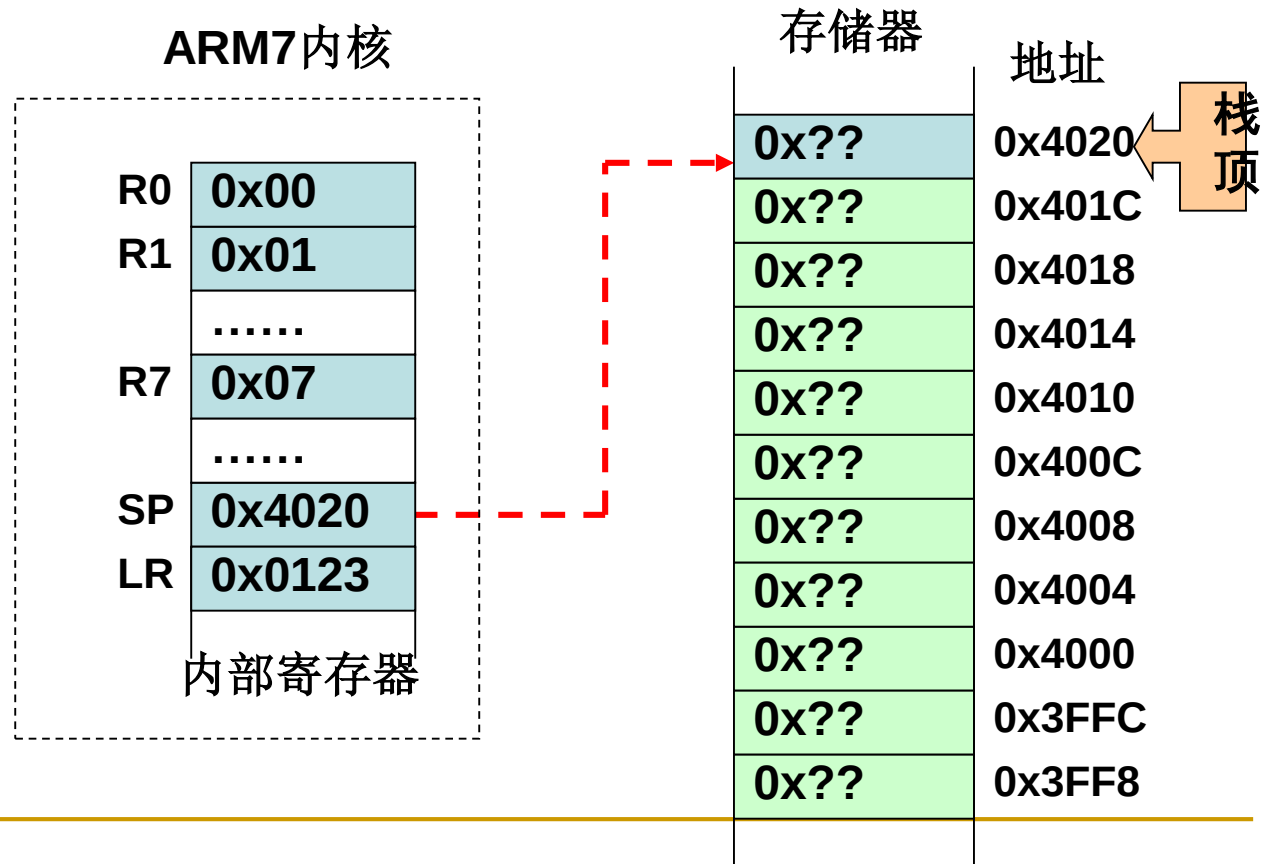
ARM存储器访问指令——满递减压栈操作

应用示例：

STMFD SP!,{R0-R7,LR}

1.压栈操作前寄存器和堆栈区的状态；

2.压栈操作前堆栈指针指向栈顶；



ARM存储器访问指令——满递减压栈操作

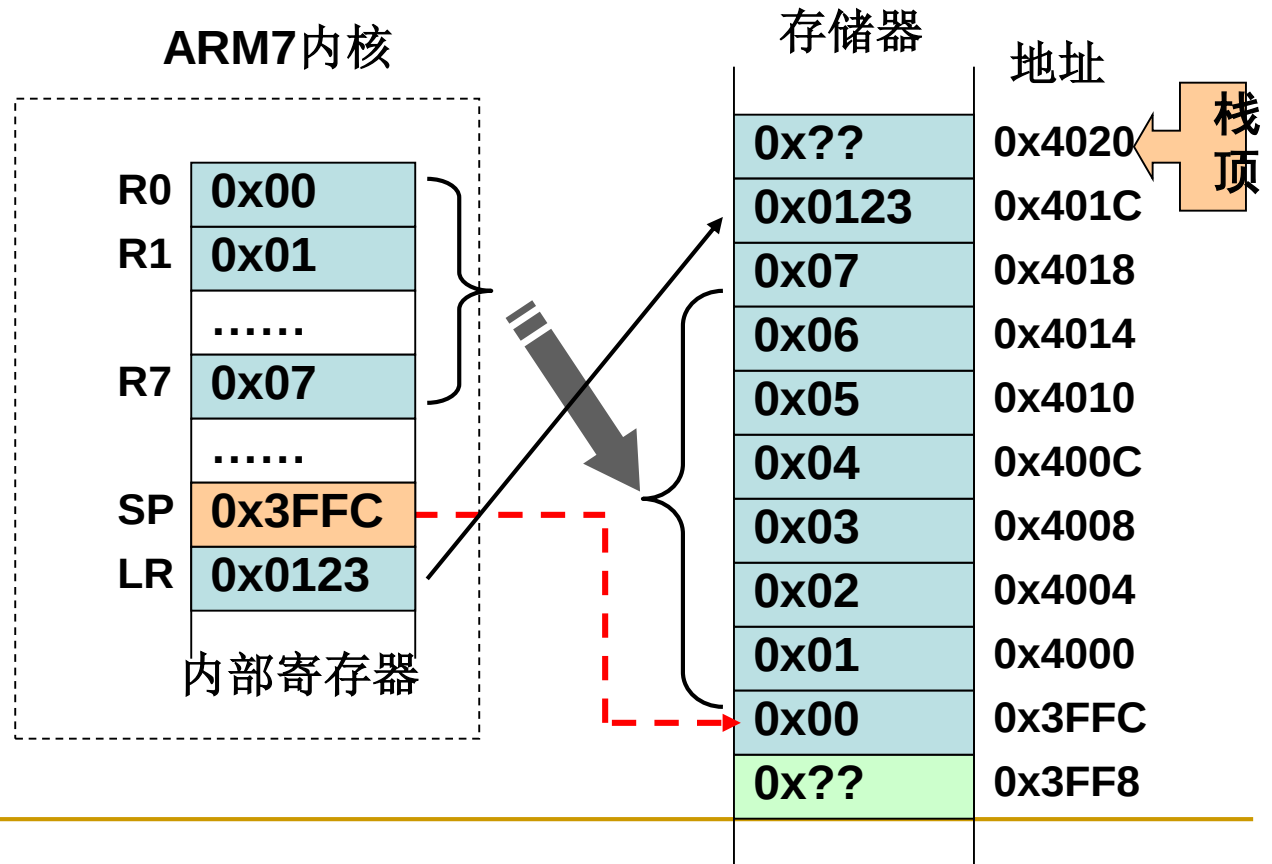
应用示例：

STMFD SP!,{R0-R7,LR}

1.压栈操作前寄存器和堆栈区的状态；

2.压栈操作前堆栈指针指向栈顶；

3.执行压栈操作指令保存R0-R7和LR



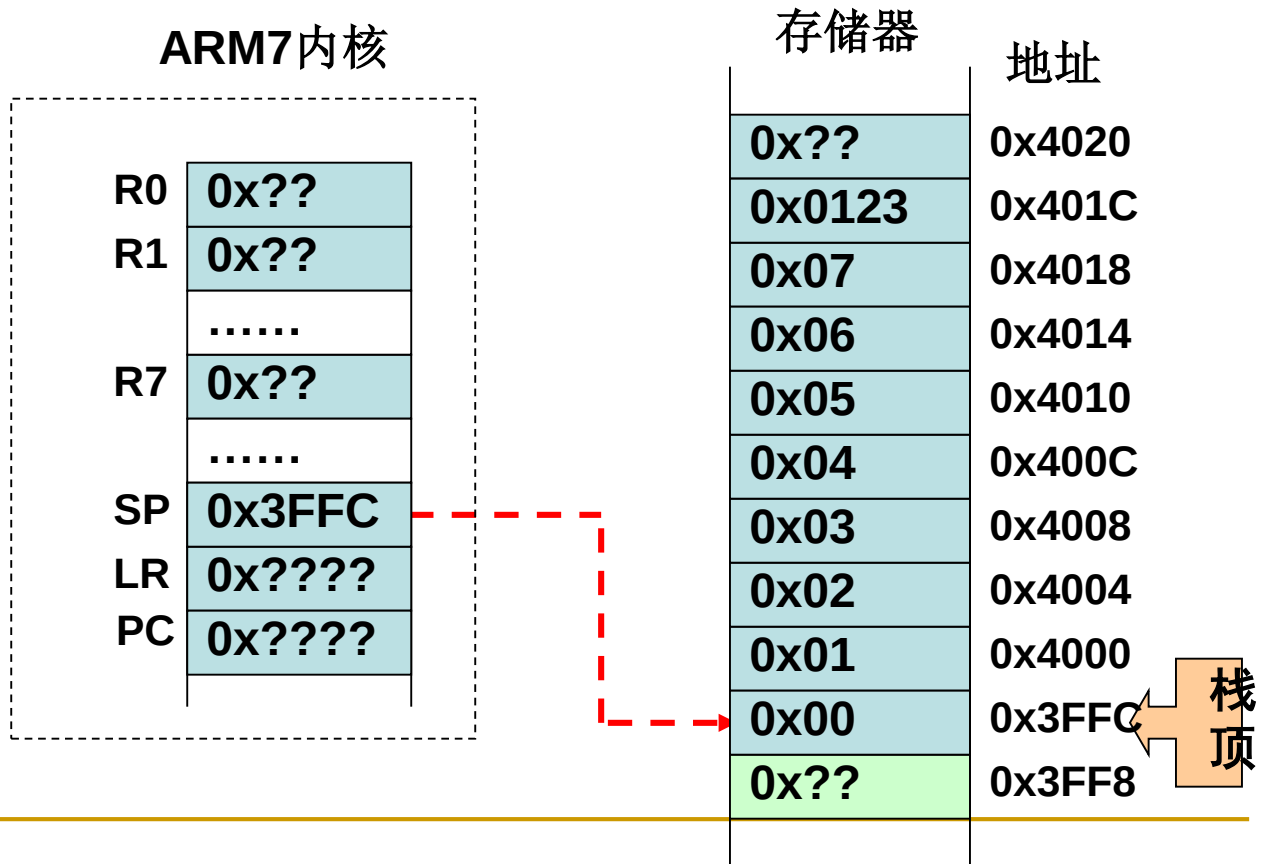
ARM存储器访问指令——满递减出栈操作

应用示例：

LDMFD SP!,{R0-R7,PC}

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；



ARM存储器访问指令——满递减出栈操作

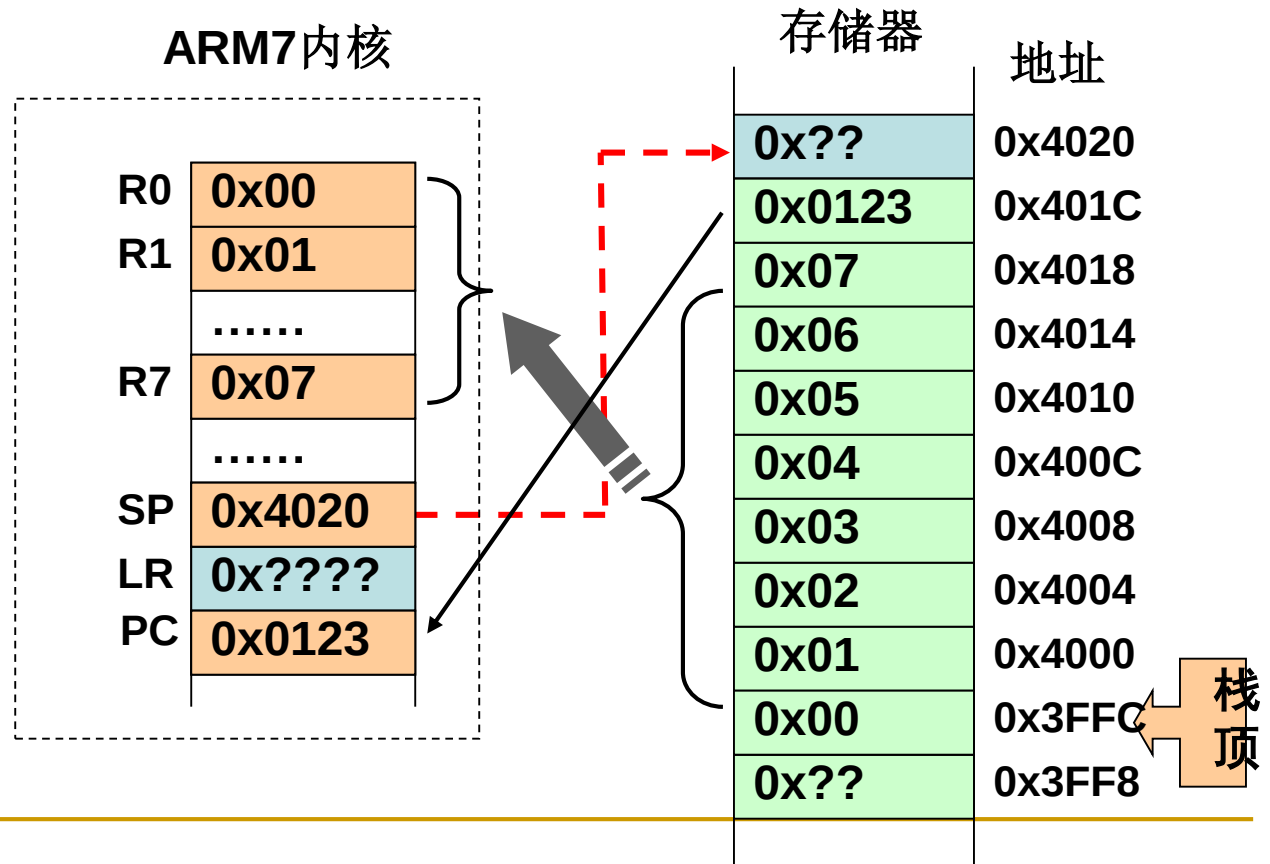
应用示例：

LDMFD SP!,{R0-R7,PC}

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；

3.执行出栈操作指令恢复R0-R7和PC



ARM存储器访问指令——满递减出栈操作

带状状态寄存器恢复的出栈操作：

LDMFD SP!,{R0-R7,PC}^

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；

ARM7内核

R0	0x??
R1	0x??
.....	
R7	0x??
.....	
SP	0x3FFC
LR	0x????
PC	0x????
CPSR	0x????
SPSR	0x????

存储器

存储器	地址
0x??	0x4020
0x0123	0x401C
0x07	0x4018
0x06	0x4014
0x05	0x4010
0x04	0x400C
0x03	0x4008
0x02	0x4004
0x01	0x4000
0x00	0x3FFC
0x??	0x3FF8

栈顶

ARM存储器访问指令——满递减出栈操作

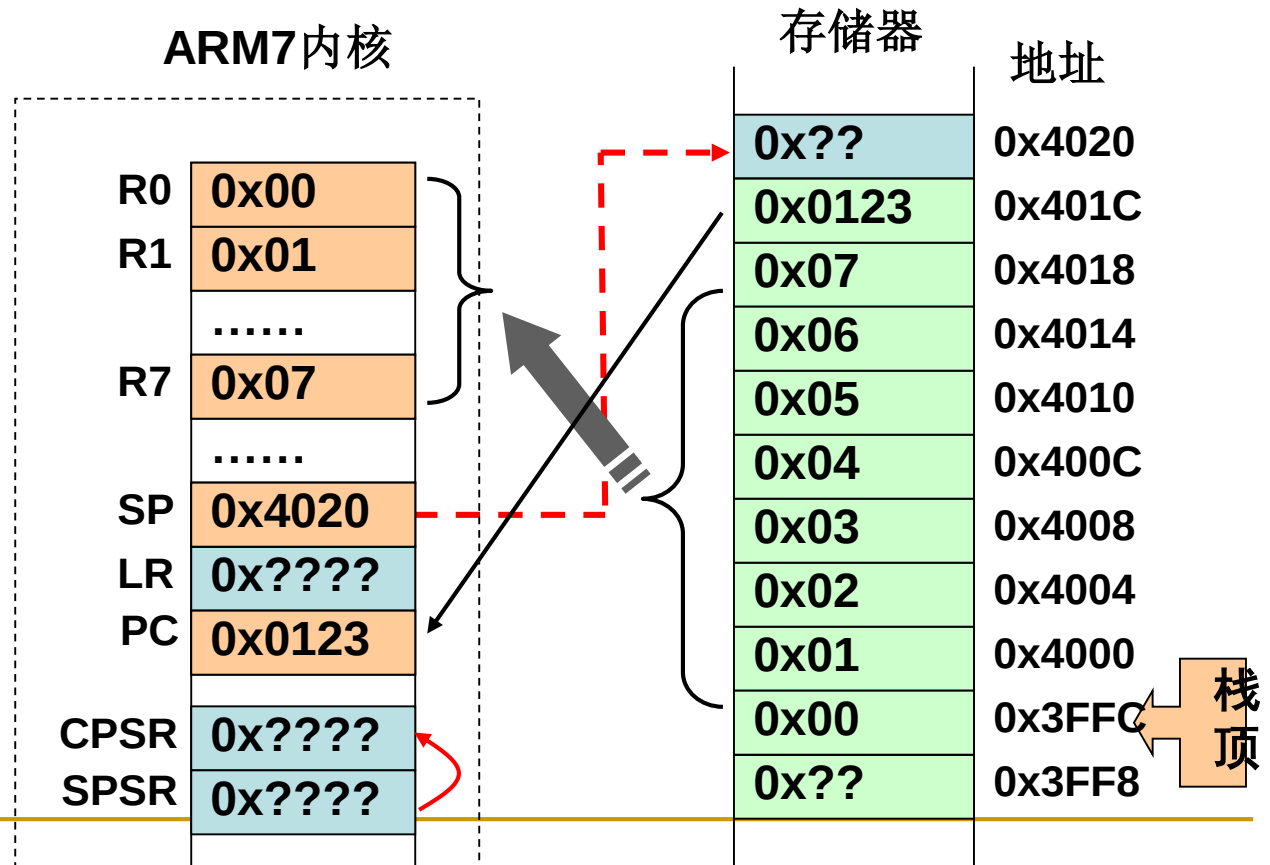
带状状态寄存器恢复的出栈操作：

LDMFD SP!,{R0-R7,PC}^

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；

3.执行出栈操作指令恢复R0-R7和PC



3.2 指令集介绍

- ARM存储器访问指令——寄存器和存储器交换指令

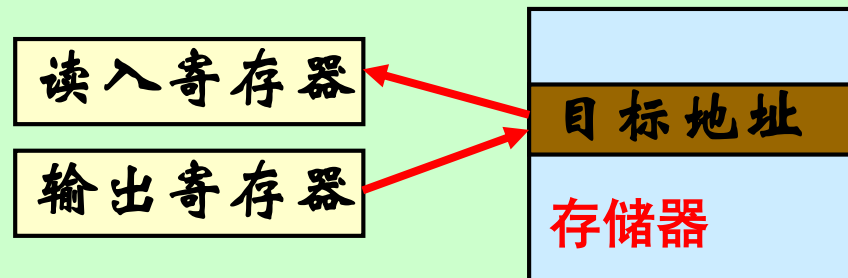
SWP指令用于将一个内存单元(该单元地址放在寄存器Rn中)的内容读取到一个寄存器Rd中, 同时将另一个寄存器Rm的内容写入到该内存单元中。使用SWP可实现信号量操作。

3.2 指令集介绍

- ARM存储器访问指令——寄存器和存储器交换指令

装载指令:

SWP 读入寄存器,输出寄存器,目标地址





3.2 指令集介绍

- ARM存储器访问指令——寄存器和存储器交换指令

装载指令:

SWP 读入寄存器,输出寄存器,目标地址

助记符	说明	操作	条件码位置
SWP Rd,Rm,Rn	寄存器和存储器 字 数据交换	$Rd \leftarrow [Rn], [Rn] \leftarrow Rm$ ($Rn \neq Rd$ 或 Rm)	SWP {cond}
SWPB Rd,Rm,Rn	寄存器和存储器 字节 数据交换	$Rd \leftarrow [Rn], [Rn] \leftarrow Rm$ ($Rn \neq Rd$ 或 Rm)	SWP {cond} B

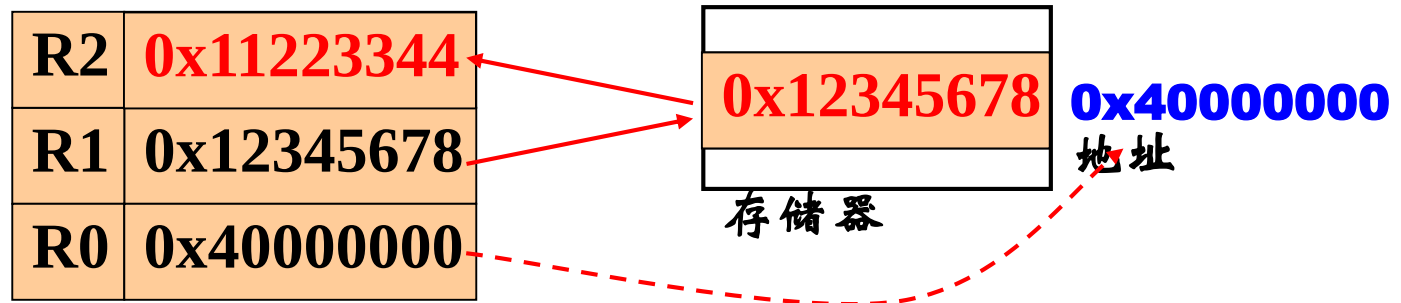
3.2 指令集介绍

- ARM存储器访问指令——寄存器和存储器交换指令

应用示例：

SWP R2,R1,[R0]

将R1的内容与R0指向的存储单元的内容进行交换



ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令

3.2 指令集介绍

■ ARM指令集——ARM数据处理指令

数据处理指令大致可分为3类：

·数据传送指令；

·算术逻辑运算指令；

·比较指令。

数据处理指令只能对寄存器的内容进行操作，而不能对内存中的数据进行操作。所有ARM数据处理指令均可选择使用S后缀，并影响状态标志。

3.2 指令集介绍

■ 数据传送指令

MOV 指令将8位图立即数或寄存器传送到目标寄存器 (Rd)，可用于移位运算等操作。

装载指令:

MOV 目标寄存器, 操作数

目标寄存器

操作数



3.2 指令集介绍

■ 数据传送指令

MOV 指令将8位图立即数或寄存器传送到目标寄存器 (Rd)，可用于移位运算等操作。

同类型的指令还有**MVN**，它可以实现数据的非传递，即把操作数取反后送至目标寄存器。

MVN 目标寄存器，操作数

目标寄存器

操作数取反



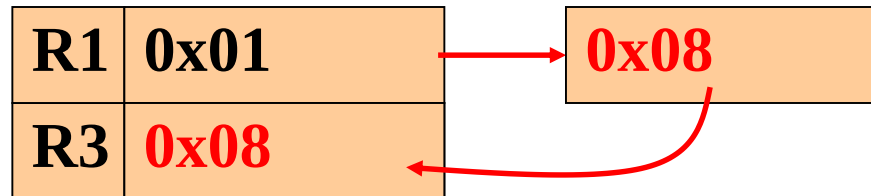
3.2 指令集介绍

■ 数据传送指令

应用示例：

MOV R3,R1,LSL #3 ;R3=R1×8

逻辑左移3位





思考与练习

1. **MOV**指令与**LDR**指令都是往目标寄存器中传送数据，但是它们有什么区别吗？



思考与练习

1. **MOV**指令与**LDR**指令都是往目标寄存器中传送数据，但是它们有什么区别吗？

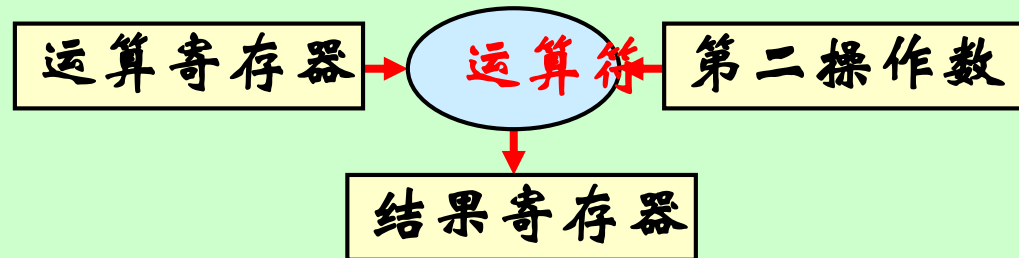
MOV指令用于将数据从一个寄存器传送到另一个寄存器中，或者将一个常数传送到一个寄存器中，但是不能访问内存。**LDR**指令用于从内存中读取数据放入寄存器中。

3.2 指令集介绍

■ 算术逻辑运算指令

算术逻辑运算指令包括“加/减”以及“与/或/异或”等指令，它们的格式如下：

OpCode 结果寄存器, 运算寄存器, 第二操作数



3.2 指令集介绍

■ 算术逻辑运算指令

OpCode

部分**算术**运算符

ADD: 加法运算

ADC: 带进位加法运算

SUB: 减法运算

RSB: 逆向减法运算

SBC: 带进位减法运算

RSC: 带进位逆向减法运算

3.2 指令集介绍

■ 算术逻辑运算指令

OpCode

部分逻辑运算符:

AND: 逻辑“与”运算

ORR: 逻辑“或”运算

EOR: 逻辑“异或”运算

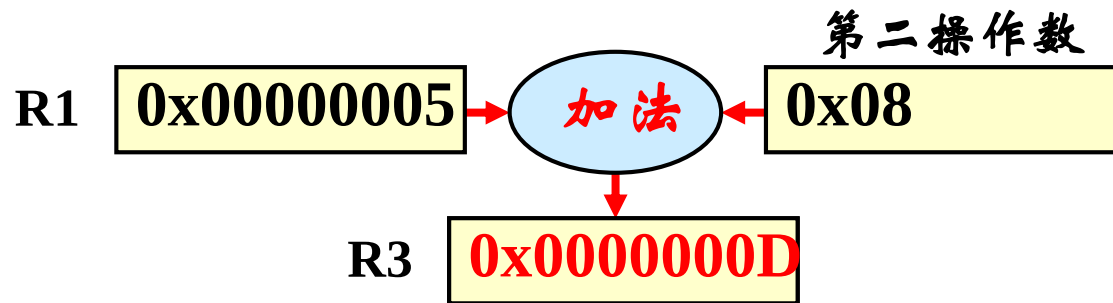
BIC: 位清除运算

3.2 指令集介绍

■ 算术逻辑运算指令

应用示例：

ADD R3,R1, #0x08 ;R3=R1+8

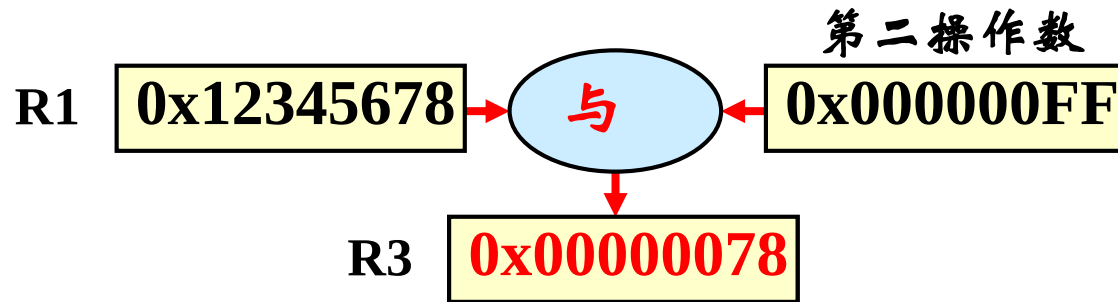


3.2 指令集介绍

■ 算术逻辑运算指令

应用示例：

AND R3,R1, #0xFF ;R3=R1 & 0x000000FF

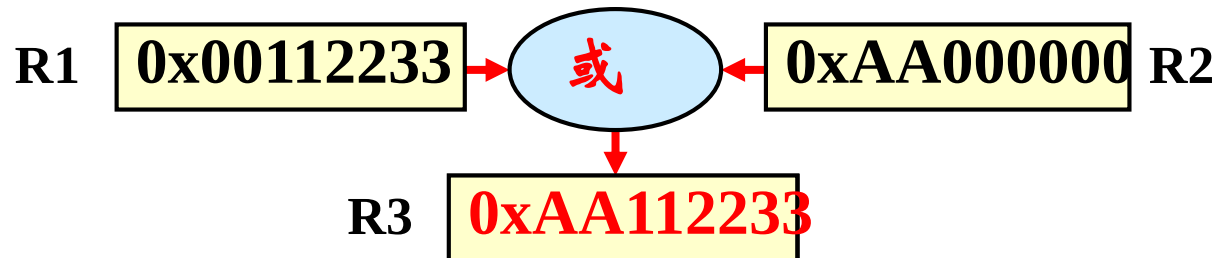


3.2 指令集介绍

■ 算术逻辑运算指令

应用示例：

ORR R3,R1, R2 ;R3=R1|R2

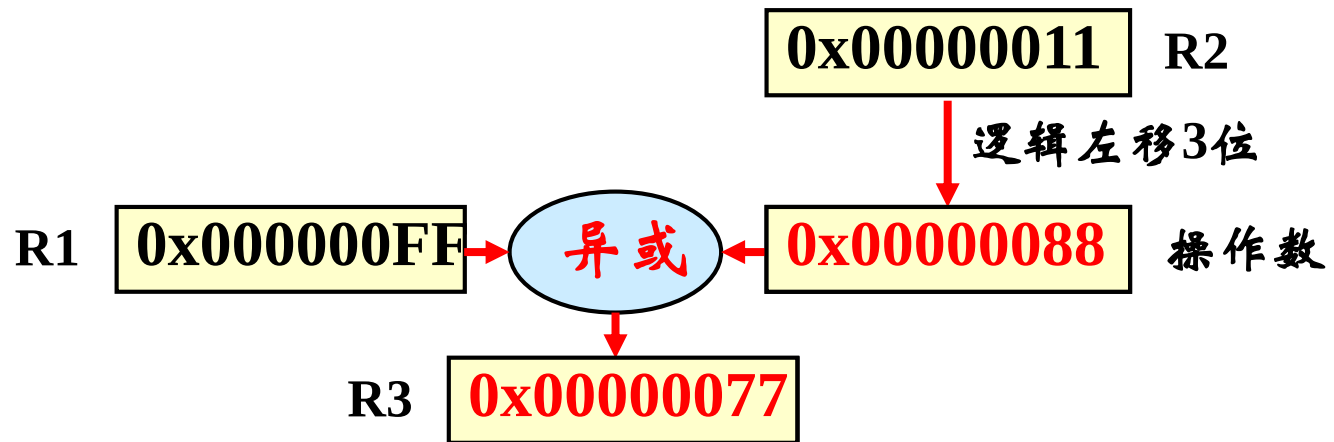


3.2 指令集介绍

■ 算术逻辑运算指令

应用示例：

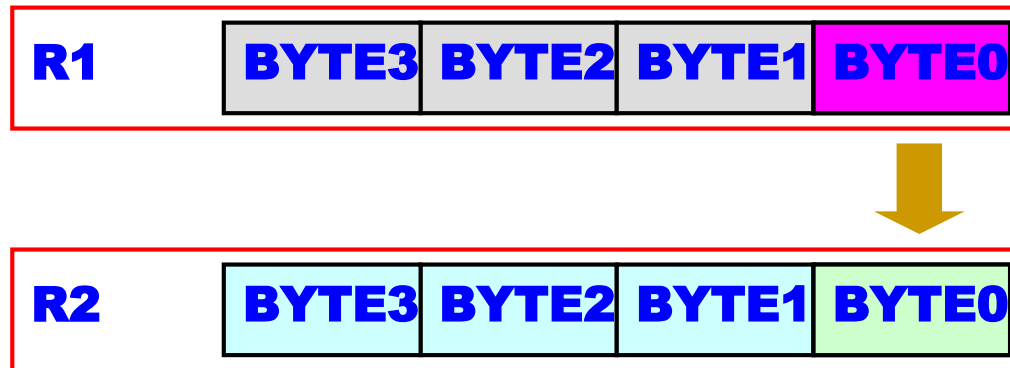
EOR R3,R1, R2,LSL 0x03 ;R3=R1 ^ (R2 × 8)





思考与练习

1. 用R1寄存器的最低字节替换掉R2寄存器的最低字节，并不影响条件标志位？





思考与练习

1. 用R1寄存器的最低字节替换掉R2寄存器的最低字节，并不影响条件标志位？

AND R1, R1, #0x000000FF

AND R2, R2, #0xFFFFFFFF00

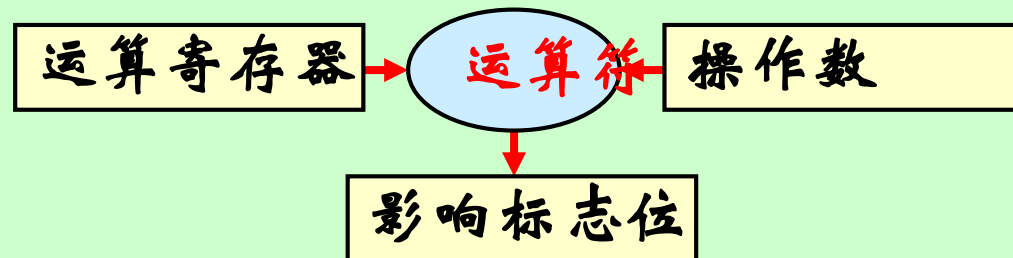
ORR R2, R2, R1

3.2 指令集介绍

■ 比较指令

比较指令将两个数值进行的特定运算，根据运算结果影响CPSR的相关标志位，用于后面程序的条件执行，但是运算结果不予保存。

OpCode 运算寄存器, 操作数





3.2 指令集介绍

■ 比较指令

OpCode

比较运算符:

CMP: 数值比较

CMN: 负数比较

TST: 位测试

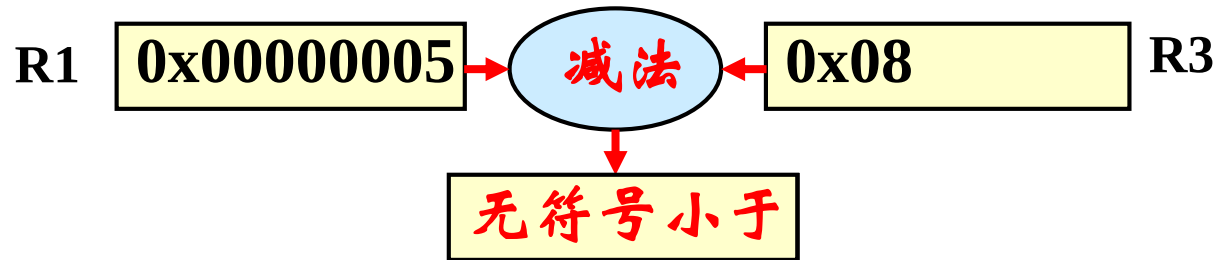
TEQ: 相等测试

3.2 指令集介绍

■ 比较指令

应用示例：

CMP R3,R1 ;R3减R1并影响标志位

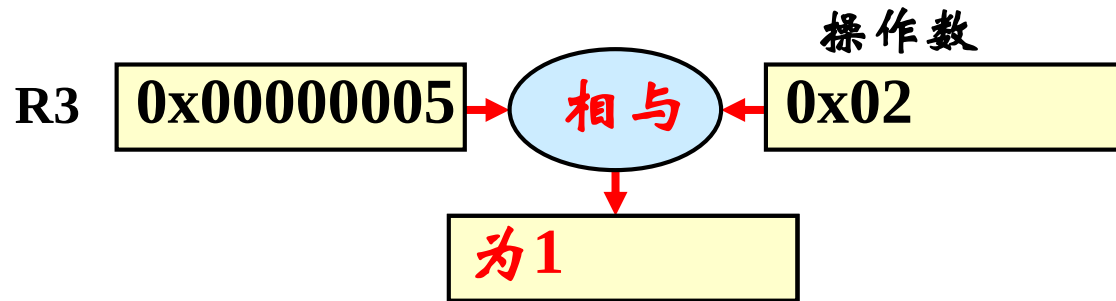


3.2 指令集介绍

■ 比较指令

应用示例：

TST R3,#0x02 ;测试R3的第2位并影响标志位

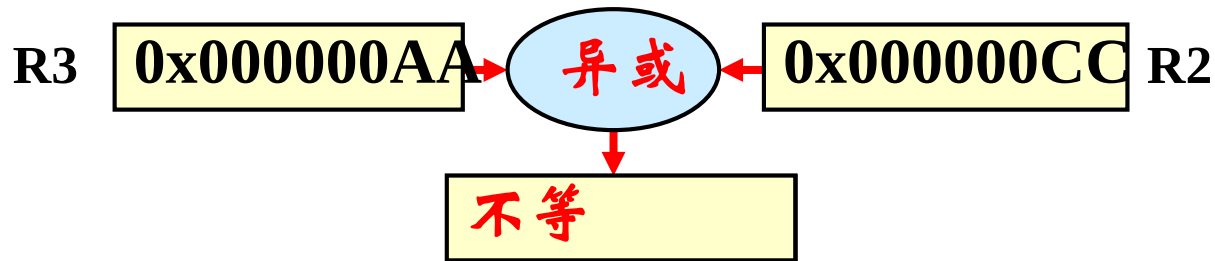


3.2 指令集介绍

■ 比较指令

应用示例：

TEQ R3,R2 ; R3与R2是否相等并影响标志位



与CMP的区别在于TEQ不影响C和V位，也就是只能判断是否相等，而不能判断是否大于，或小于。

ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令



3.2 指令集介绍

■ ARM指令集——乘法指令

ARM7TDMI具有三种乘法指令，分别为：

- 32×32 位乘法指令；
- 32×32 位乘加指令；
- 32×32 位结果为64位的乘/乘加指令。



3.2 指令集介绍

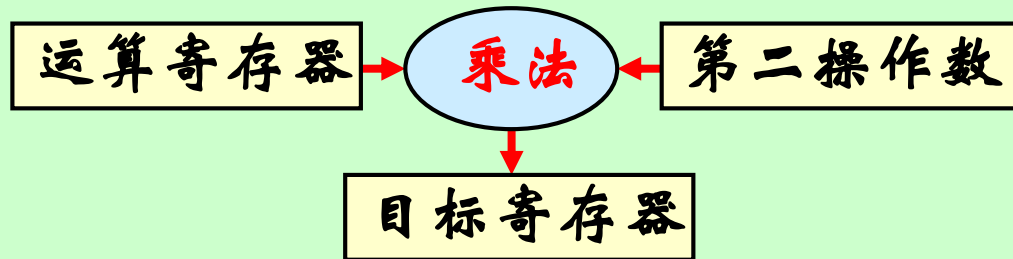
■ ARM指令集——乘法指令

助记符	说明	操作	条件码位置
MUL Rd,Rm,Rs	32位乘法指令	$Rd \leftarrow Rm * Rs$ ($Rd \neq Rm$)	MUL {cond} {S}
MLA Rd,Rm,Rs,Rn	32位乘加指令	$Rd \leftarrow Rm * Rs + Rn$ ($Rd \neq Rm$)	MLA {cond} {S}
UMULL RdLo,RdHi,Rm,Rs	64位无符号乘法指令	$(RdLo, RdHi) \leftarrow Rm * Rs$	UMULL {cond} {S}
UMLAL RdLo,RdHi,Rm,Rs	64位无符号乘加指令	$(RdLo, RdHi) \leftarrow Rm * Rs + (RdLo, RdHi)$	SMLAL {cond} {S}
SMULL RdLo,RdHi,Rm,Rs	64位有符号乘法指令	$(RdLo, RdHi) \leftarrow Rm * Rs$	SMULL {cond} {S}
SMLAL RdLo,RdHi,Rm,Rs	64位有符号乘加指令	$(RdLo, RdHi) \leftarrow Rm * Rs + (RdLo, RdHi)$	SMLAL {cond} {S}

3.2 指令集介绍

■ ARM指令集—— 32×32 位乘法指令

MUL 目标寄存器, 运算寄存器, 第二操作数

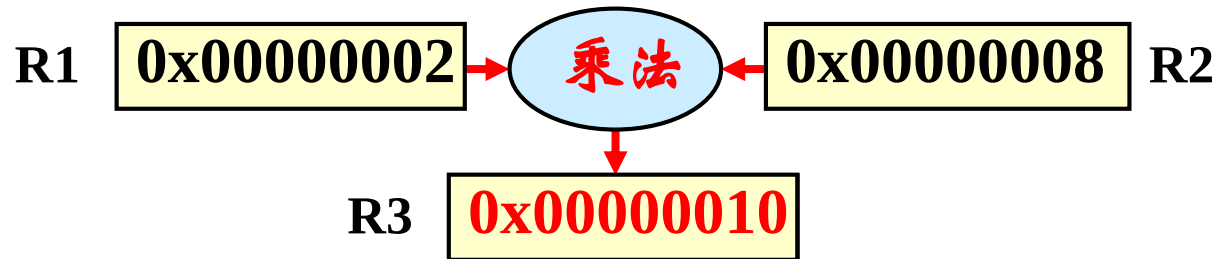


3.2 指令集介绍

■ ARM指令集—— 32×32 位乘法指令

应用示例：

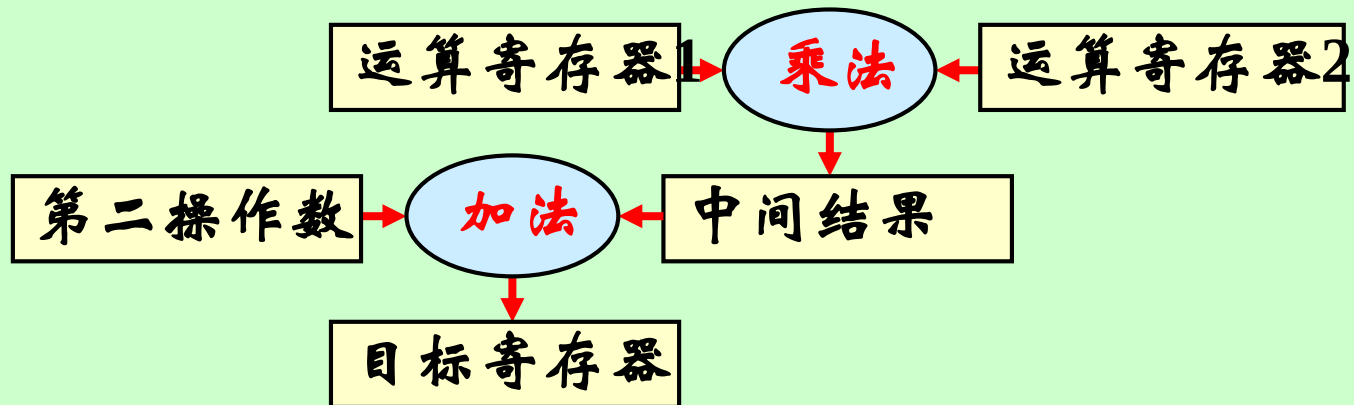
MUL R3,R2,R1 ; $R3 = R2 \times R1$



3.2 指令集介绍

■ ARM指令集—— 32×32 位乘加法指令

MLA 目标寄存器, 运算寄存器1, 运算寄存器2, 第二操作数

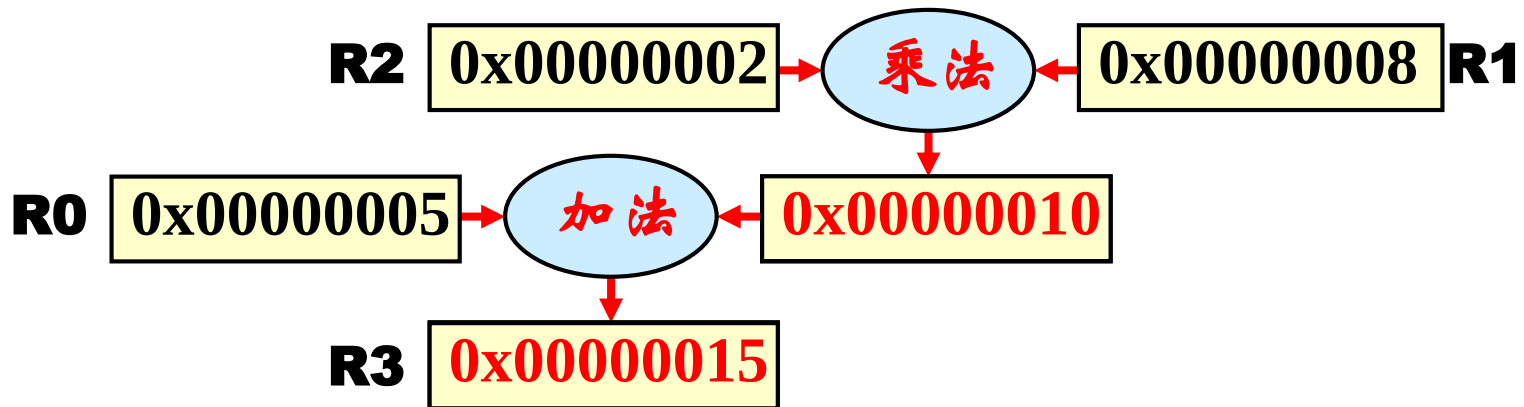


3.2 指令集介绍

■ ARM指令集——32×32位乘加法指令

应用示例：

MLA R3,R2,R1,R0 ; $R3 = R2 \times R1 + R0$



ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令



3.2 指令集介绍

■ ARM指令集——程序如何跳转

在ARM中有两种方式可以实现程序的跳转：

1.直接向PC寄存器赋值实现跳转；

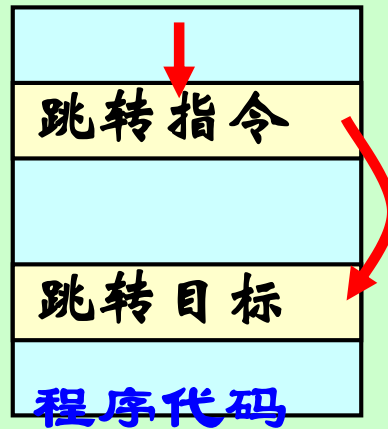
例： MOV PC,R14

2.使用分支指令直接跳转。

3.2 指令集介绍

■ ARM指令集—— 分支指令

OpCode **跳转目标**



3.2 指令集介绍

■ ARM指令集—— 分支指令

OpCode **跳转目标**

分支指令种类：

B：分支指令

BL：带链接的分支指令

BX：带状态切换的分支指令

• ARM指令——分支指令

B/BL指令编码格式

31	28	27	26	25	24	23	0
cond		1	0	1	L	signed_immed_24	

指令执行的
条件码

操作码表示区

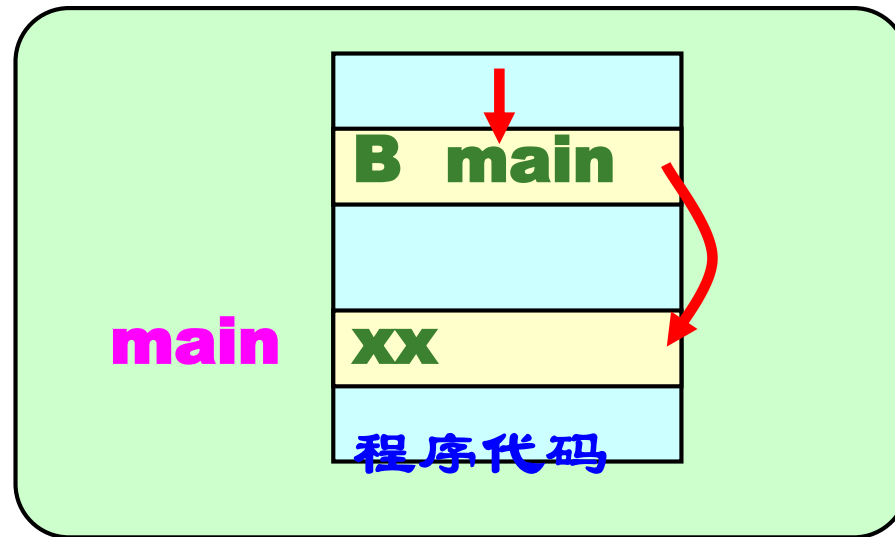
24位常数，跳转目标地址
(基于PC的偏移量)

因为用来表示目标地址的位数有限，所以B/BL指令无法实现4G（32位）范围内的任意跳转

3.2 指令集介绍

■ ARM指令集—— 分支指令“B”

B指令为简单的跳转指令，不附带其它功能。跳转范围限制在当前指令的 $\pm 32\text{M}$ 字节地址内(ARM指令为字对齐，最低2位地址固定为0)。

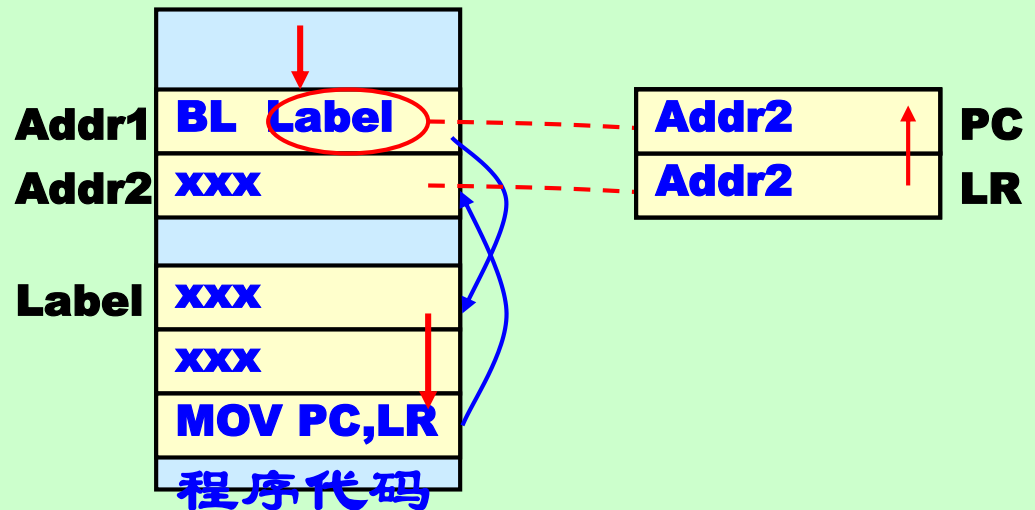


ARM指令——分支指令“BL”

带链接的分支指令——BL指令除了具有跳转功能，还能在跳转之前将下一条指令的地址拷贝到R14(即LR) 链接寄存器中，**它适用于子程序调用**。跳转范围限制在当前指令的 $\pm 32\text{M}$ 字节地址内。指令格式如下：

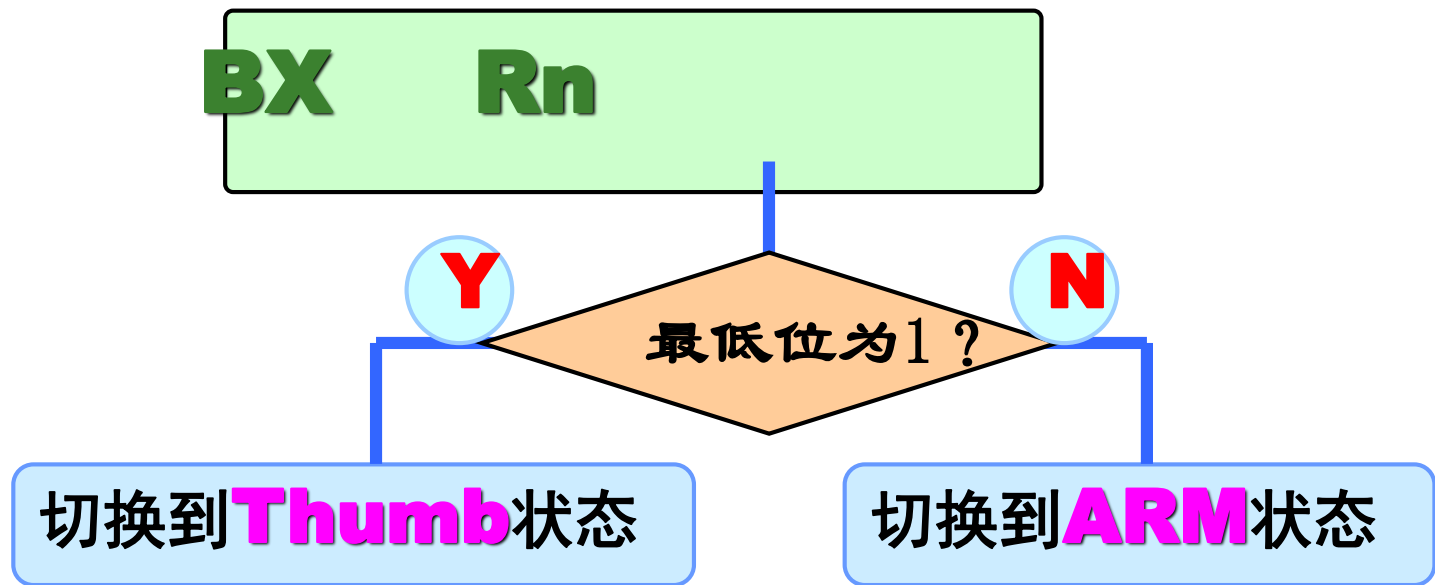
例如： **BL DelayNS ;调用子程序DelayNS**

2. 程序跳转到目标地址Label继续执行，当子程序执行结束后，将LR寄存器内容存入PC，返回调用函数继续执行



• ARM指令——分支指令“BX”

带状态切换的分支指令——BX指令除了具有跳转功能，还能在跳转的同时切换处理器状态。其跳转范围不受限制。指令格式如下：



• ARM指令——分支指令 **“BX”**

带状态切换的分支指令——**BX**指令除了具有跳转功能，还能**在跳转的同时切换处理器状态**。其跳转范围不受限制。指令格式如下：

BX **Rn**

例如：

ADRL R0,T_Fun+1 ;将Thumb程序的入口地址加1存入R0
BX R0 ;跳转到R0指定的地址，
 ;并根据R0的最低位来切换处理器状态



ARM指令种类

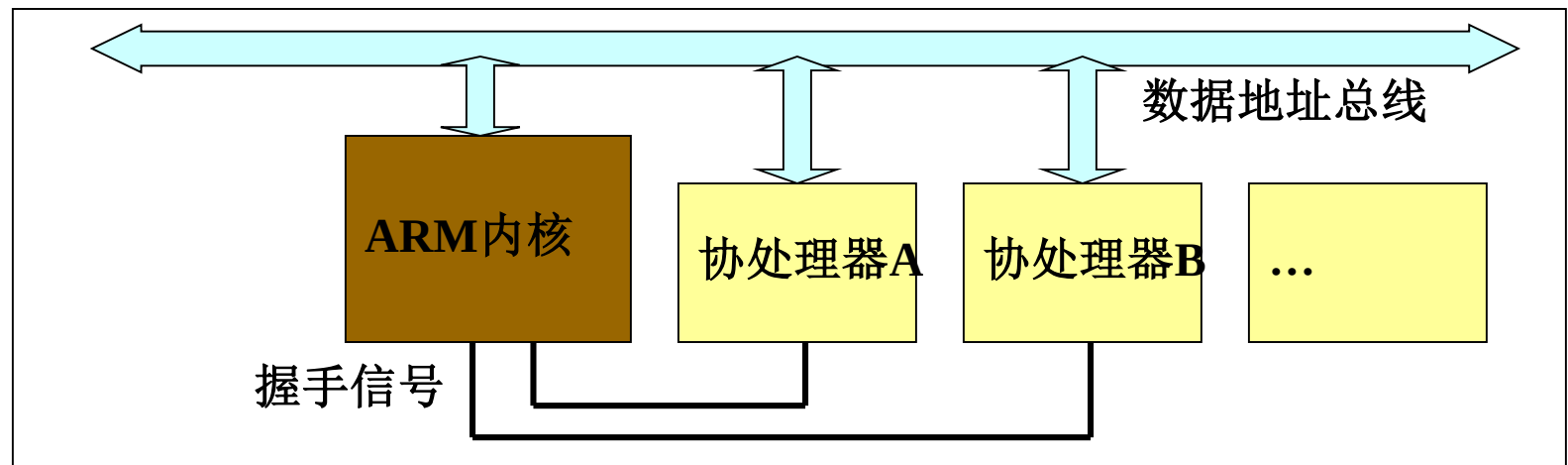
1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令

3.2 指令集介绍

■ ARM指令集——协处理器指令

ARM内核支持协处理器操作，协处理器的控制要通过协处理器命令实现。

ARM内核与协处理器的关系





• ARM指令——协处理器指令

助记符	说明	操作	条件码位置
CDP coproc, opcode1, CRd, CRn, CRm{, opcode2}	协处理器数据操作指令	取决于协处理器	CDP {cond}
LDC {L} coproc, CRd, <地址>	协处理器数据读取指令	取决于协处理器	LDC {cond} {L}
STC {L} coproc, CRd, <地址>	协处理器数据写入指令	取决于协处理器	STC {cond} {L}
MCR coproc, opcode1, Rd, CRn, CRm{, opcode2}	ARM寄存器到协处理器寄存器的数据传送指令	取决于协处理器	MCR {cond}
MRC coproc, opcode1, Rd, CRn, CRm{, opcode2}	协处理器寄存器到ARM寄存器到的数据传送指令	取决于协处理器	MCR {cond}

ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令



3.2 指令集介绍

■ ARM指令集——杂项指令

在ARM指令集中杂项指令共有3条，它们非常重要，特别是与操作系统的使用息息相关：

1. 软件中断产生指令： **SWI**
2. 程序状态寄存器读指令： **MRS**
3. 程序状态寄存器写指令： **MSR**

3.2 指令集介绍

■ ARM指令集——软中断指令 “SWI”

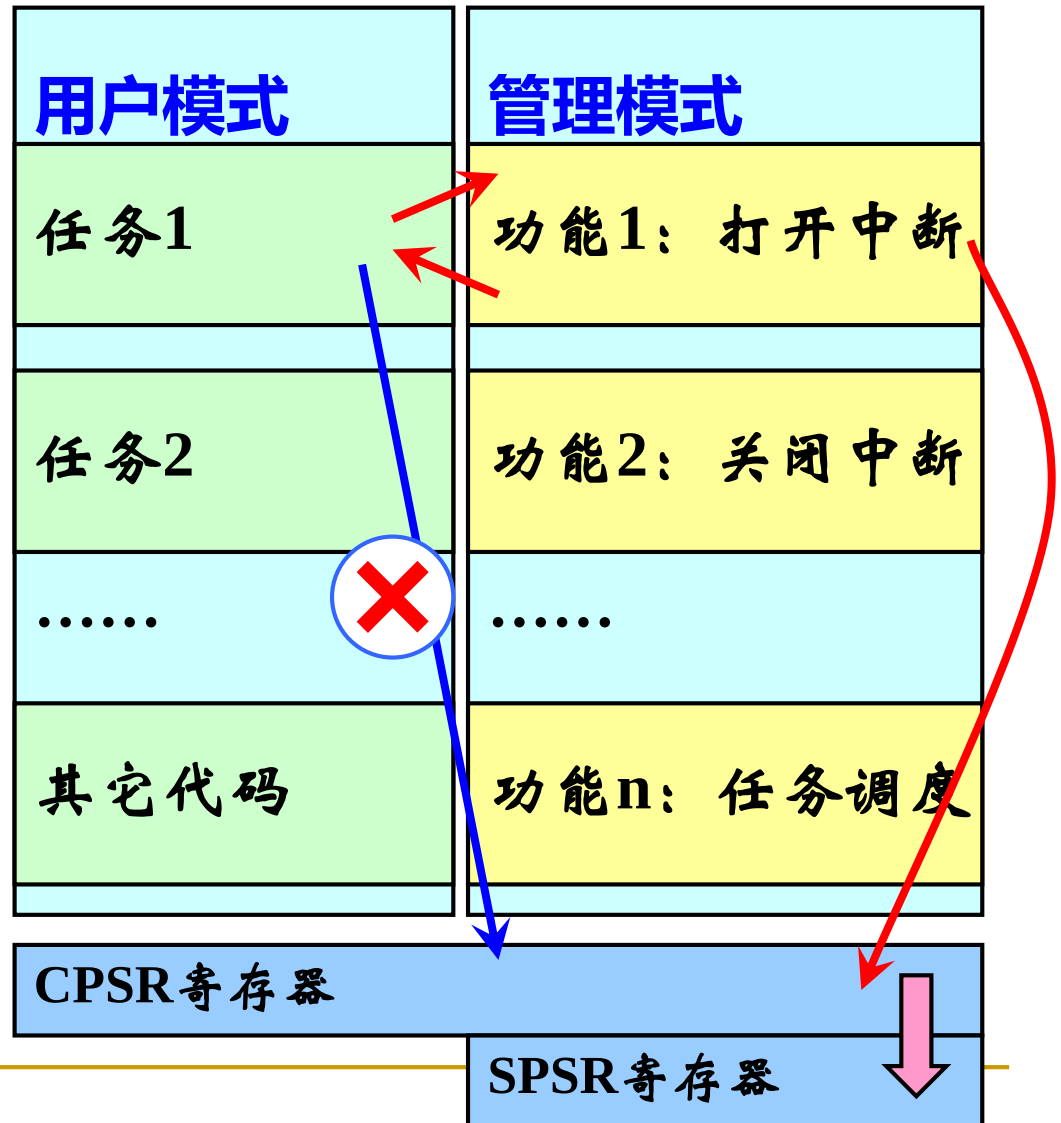
SWI指令用于产生软中断，主要用于用户程序调用操作系统的系统服务。执行该指令后，处理器将完成以下动作：

1. 切换到**管理模式**
2. 将CPSR备份到管理模式下的SPSR寄存器
3. 程序跳转到软件中断入口

ARM杂项指令——软中断指令

1.使用操作系统后，为了防止出错的任务影响其它任务的执行，通常将任务放在用户模式执行，以限制其权限；

2.对于一些重要的操作，如中断的开关，必须由操作系统完成。使用软件中断指令即可完成系统功能调用；



3.2 指令集介绍

■ ARM指令集——软中断指令 **“SWI”**

SWI指令格式如下所示：

SWI immed_24

可以利用指令中的这个24位立即数向软中断服务程序传递参数，比如请求的服务类型



• ARM杂项指令——软中断指令

根据SWI指令传递的参数SWI异常处理程序可以作出相应的处理。SWI指令传递参数有以下两种方法，

- 指令中的24位立即数指定了用户请求的服务类型，参数通过通用寄存器传递。

MOV R0, #34 ;设置子功能号为34

SWI 12 ;调用12号软中断

- 指令中的24位立即数被忽略，用户请求的服务类型由寄存器R0的值决定，参数通过其它的通用寄存器传递。

MOV R0, #12 ;调用12号软中断

MOV R1, #34 ;设置子功能号为34

SWI 0

ARM杂项指令——软中断指令

在SWI异常中断处理程序中，取出SWI指令中立即数的步骤为：

- 首先确定引起软中断的SWI指令是ARM指令还是Thumb指令，这可通过对SPSR访问得到；
- 然后取得该SWI指令的地址，这可通过访问LR寄存器得到；
- 接着读出该SWI指令，分解出立即数。

SWI_Handler

STMFD SP!, {R0-R3, R12, LR} ; 现场保护

MRS R0, SPSR ; 读取SPSR

STMFD SP!, {R0} ; 保存SPSR

TST R0, #0x20 ; 测试T标志位

LDRNEH R0, [LR, #-2] ; 若是Thumb指令，读取指令码(16位)

BICNE R0, R0, #0xFF00 ; 取得Thumb指令的8位立即数

LDREQ R0, [LR, #-4] ; 若是ARM指令，读取指令码(32位)

BICEQ R0, R0, #0xFF000000 ; 取得ARM指令的24位立即数

...

LDMFD SP!, {R0-R3, R12, PC} ; SWI异常中断返回

3.2 指令集介绍

■ ARM指令集——状态寄存器读指令 **“MRS”**

在ARM处理器中，只有MRS指令可以对状态寄存器CPSR和SPSR进行读操作。通过读CPSR可以了解当前处理器的工作状态。读SPSR寄存器可以了解到进入异常前的处理器状态。指令格式如下所示：

MRS 目标寄存器, **psr**

目标寄存器

PSR



3.2 指令集介绍

■ ARM指令集——状态寄存器读指令 “MRS”

应用示例：

MRS R1,CPSR ; 读取CPSR状态寄存器到R1

MRS R2,SPSR ; 读取SPSR状态寄存器到R2



ARM杂项指令——状态寄存器写指令“MSR”

在ARM处理器中，只有MSR指令可以对状态寄存器CPSR和SPSR进行写操作。与MRS配合使用，可以实现对CPSR或SPSR寄存器的读-修改-写操作，可以切换处理器模式等操作。

MSR psr_field, 操作数

PSR寄存器被分为四个8位的域：

1. 状态位域：用**s**表示
2. 扩展位域：用**x**表示
3. 条件标志位域：用**f**表示
4. 控制位域：用**c**表示

操作数分为两种：

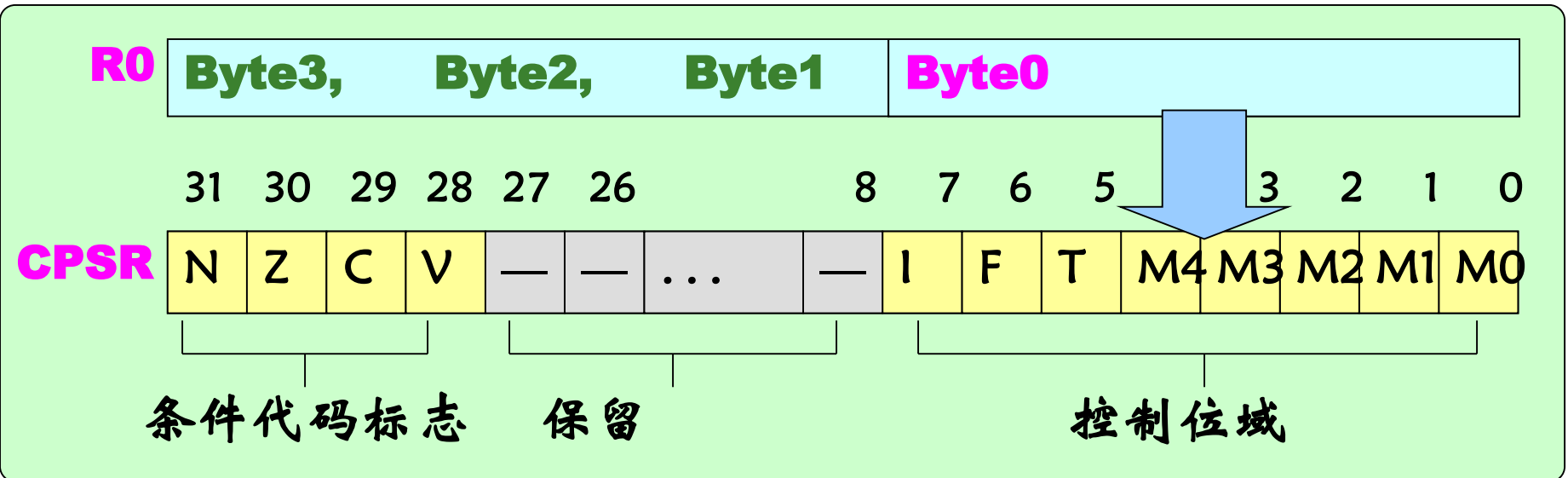
1. 寄存器
2. 8位图立即数

3.2 指令集介绍

■ ARM指令集——状态寄存器写指令 “MSR”

示例，将R0的内容写入CPSR寄存器的控制位域

MSR CPSR_c,R0



ARM杂项指令——状态寄存器写指令“MSR”

启动代码堆栈初始化应用示例：

INITSTACK

MOV R0, LR

;设置管理模式堆栈

MSR CPSR_C, #0xD3

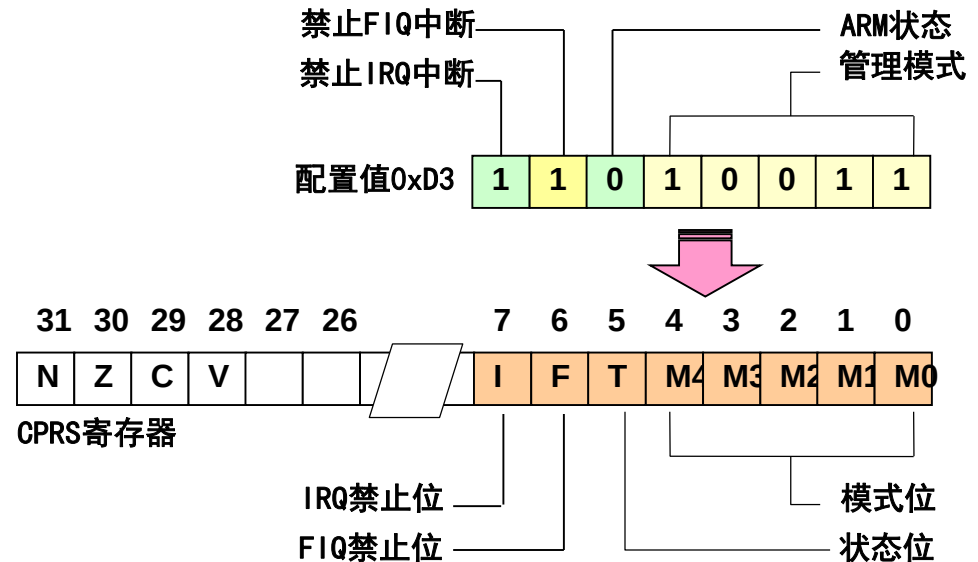
LDR SP, StackSvc

;设置中断模式堆栈

MSR CPSR_C, #0xD2

LDR SP, StackIrq

.....



ARM杂项指令——状态寄存器写指令“MSR”

启动代码堆栈初始化应用示例：

INITSTACK

MOV R0, LR

;设置管理模式堆栈

MSR CPSR_C, #0xD3

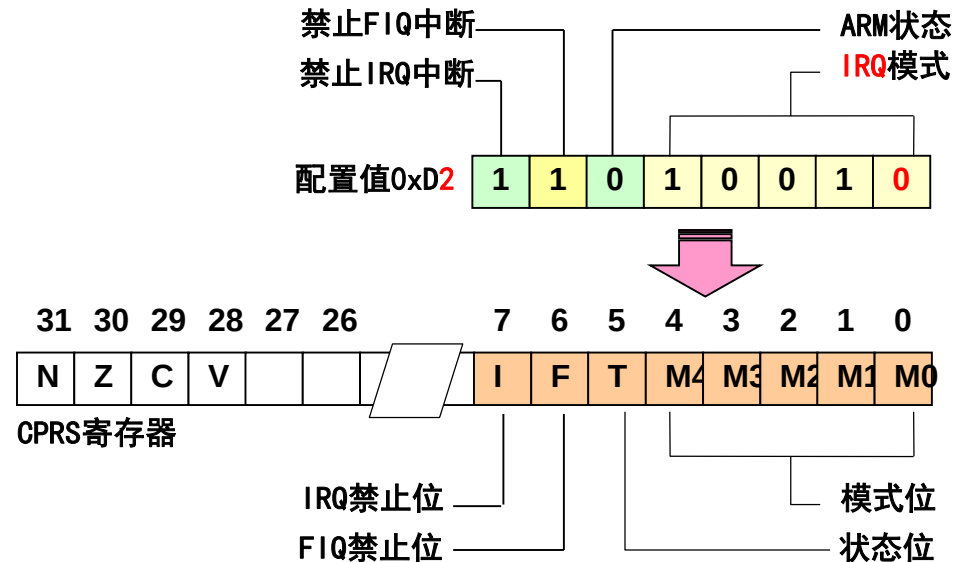
LDR SP, StackSvc

;设置中断模式堆栈

MSR CPSR_C, #0xD2

LDR SP, StackIrq

.....





思考与练习

1. 使用**MSR**和**MRS**指令，通过修改**CPSR**寄存器，实现打开/关闭**IRQ**中断的两个子程序，注意不能影响其它位？



思考与练习

;子程序：使能IRQ中断

Enable_IRQ

MRS R0, CPSR

BIC R0, R0, #0x80

MSR CPSR_c, R0

MOV PC, LR

1.将CPSR寄存器内容读出到R0;

2.修改对应于CPSR中的I控制位;

;子程序：禁能IRQ中断

Disable_IRQ

MRS R0, CPSR

ORR R0, R0, #0x80

MSR CPSR_c, R0

MOV PC, LR

3.将修改后的值写回CPSR寄存器的对应控制域;

4.返回上一层函数;

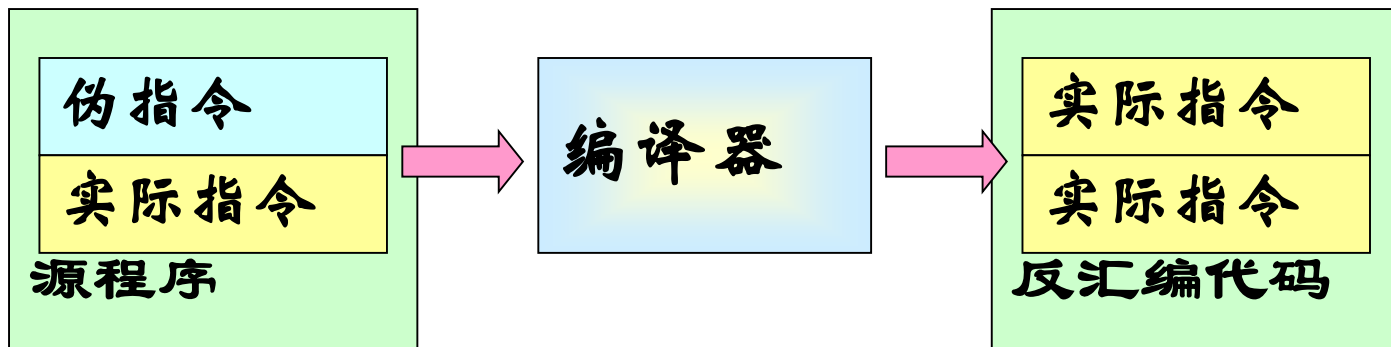
ARM指令种类

1. 存储器访问指令
2. 数据处理指令
3. 乘法指令
4. ARM分支指令
5. 协处理器指令
6. 杂项指令
7. 伪指令

3.2 指令集介绍

■ ARM伪指令

ARM伪指令不属于ARM指令集中的指令，是为了编程方便而定义的。伪指令可以像其它ARM指令一样使用，但在编译时这些指令将被等效的ARM指令代替。



3.2 指令集介绍

■ ARM伪指令

ARM伪指令有四条：

1. 小范围地址读取指令：**ADR**
2. 中等范围地址读取指令：**ADRL**
3. 大等范围地址读取指令：**LDR**
4. 空操作指令：**NOP**

• ARM伪指令——小范围的地址读取

ADR伪指令将基于PC相对偏移的地址值或基于寄存器相对偏移的地址值读取到寄存器中。在汇编编译器编译源程序时，ADR伪指令被编译器替换成一条合适的指令，若不能用一条指令实现，则产生错误，编译失败。

ADR 目标寄存器,表达式

目标寄存器



表达式

• ARM伪指令——小范围的地址读取

地址表达式expr的取指范围：

- 当地址值不是字对齐时，其取指范围为 **± 255** ；
- 当地址值是字对齐时，其取指范围为 **± 1020** ；
- 当地址值是16字节对齐时，其取指范围将更大。

ADR **目标寄存器,表达式**

目标寄存器



表达式

ARM伪指令——小范围的地址读取

应用示例（源程序）：

```
...  
ADR    R0,Delay  
...  
Delay  
MOV    R0,r14  
...
```

使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x20  ADD    r0,pc,#0x3c  
...  
...  
0x64  MOV    r0,r14  
...
```

地址

程序代码

• ARM伪指令——小范围的地址读取

应用示例（源程序）：

```
...  
ADR    R0,Delay  
...  
Delay  
MOV    R0,r14  
...
```

使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x20   ADD    r0,pc,#0x3c  
...  
...  
0x64   MOV    r0,r14  
...
```

ADR伪指令被汇编成一条指令

• ARM伪指令——小范围的地址读取

；查表应用示例：

ADR R0,DISP_TAB ；加载转换表地址

LDRB R1,[R0,R2] ；使用R2作为参数，进行查表

...

DISP_TAB

DCB 0xC0,0xF9,0xA4,0xB0,0x99, 0x92,0x82,0xF8

• ARM伪指令——中等范围的地址读取

ADRL伪指令将基于PC相对偏移的地址值或基于寄存器相对偏移的地址值读取到寄存器中，比ADR伪指令可以读取更大范围的地址。在汇编编译器编译源程序时，ADRL伪指令被编译器替换成两条合适的指令。若不能用两条指令实现，则产生错误，编译失败。

ADRL 标寄存器,表达式

目标寄存器



表达式

• ARM伪指令——中等范围的地址读取

地址表达式expr的取指范围：

- 当地址值不是字对齐时，其取指范围为 **$\pm 64\text{K}$**
- 当地址值是字对齐时，其取指范围为 **$\pm 256\text{K}$**
- 当地址值是16字节对齐时，其取指范围将更大

ADRL

标寄存器,表达式

目标寄存器



表达式

• ARM伪指令——中等范围的地址读取

应用示例（源程序）：

```
...  
ADRL R0,Delay  
...  
Delay  
MOV R0,r14  
...
```

使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x20 ADD r0,pc,#40  
0x24 ADD r0,r0,#0  
...  
0x68 MOV r0,r14  
...
```

地址

程序代码

• ARM伪指令——中等范围的地址读取

应用示例（源程序）：

```
...  
ADRL  R0,Delay  
...  
Delay  
MOV   R0,r14  
...
```

使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x20  ADD   r0,pc,#40  
0x24  ADD   r0,r0,#0  
...  
0x68  MOV   r0,r14  
...
```

ADRL伪指令被汇编成两条指令，
尽管第2条指令并没有意义

• ARM伪指令——大范围的地址读取

LDR伪指令用于加载32位的立即数或一个地址值到指定寄存器。在汇编编译源程序时，LDR伪指令被编译器替换成一条合适的指令。若加载的常数未超出MOV或MVN的范围，则使用MOV或MVN指令代替该LDR伪指令，否则汇编器将常量放入文字池，并使用一条程序相对偏移的LDR指令从文字池读出常量。

LDR 标寄存器, = 表达式

目标寄存器



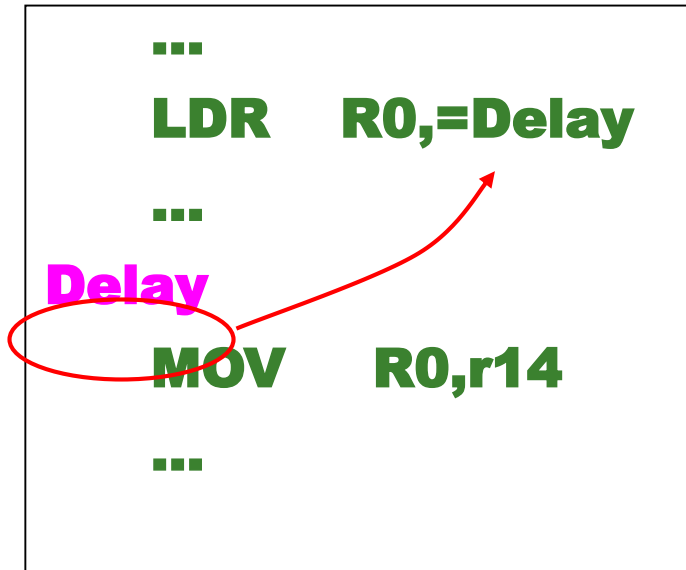
表达式

地址表达式expr的取指范围为任意值

• ARM伪指令——大范围的地址读取

应用示例（源程序）：

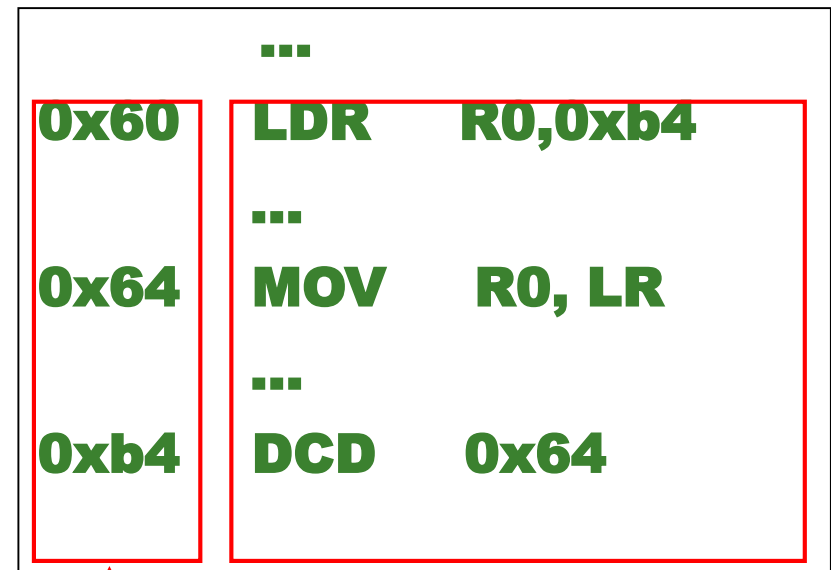
```
...  
LDR R0,=Delay  
...  
Delay  
MOV R0,r14  
...
```



使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x60 LDR R0,0xb4  
...  
0x64 MOV R0, LR  
...  
0xb4 DCD 0x64
```



地址

程序代码

ARM伪指令——大范围的地址读取

应用示例（源程序）：

```
...  
LDR R0,=Delay  
...  
Delay  
MOV R0,r14  
...
```

必须加入“=”

使用伪指令将程序标号Delay的地址存入R0

编译后的反汇编代码：

```
...  
0x60 LDR R0,0xb4  
...  
0x64 MOV R0, LR  
...  
0xb4 DCD 0x64  
...
```

必须小于4KB

LDR伪指令被汇编成一条LDR指令，并在文字池中定义了一个常量，该常量为Delay标号的地址

3.2 指令集介绍

■ ARM伪指令——空操作伪指令 **“NOP”**

NOP伪指令在汇编时将会被代替成ARM中的空操作，比如可能是“MOV R0,R0”指令等。**NOP**可用于延时操作。

指令格式：

NOP



思考与练习

1. 请使用**NOP**伪指令、**比较指令**、**条件跳转指令**等完成一个软件延时子程序，延时长度由**R0**寄存器的数值控制？



思考与练习

1. 请使用**NOP**伪指令、比较指令、条件跳转指令等完成一个软件延时子程序，延时长度由**R0**寄存器的数值控制？

Delay

NOP ;空操作

NOP

NOP

SUBS R0,R0,#1 ;循环次数减一

BNE Delay ;如果循环未结束,跳转**Delay**继续

MOV PC,LR ;子程序返回

本章学习重点

1. ARM处理器的寻址方式

——共有9种寻址方式；

2. ARM指令的特点

——可条件执行、可选择影响标志位、具有非常灵活的第二操作数；

3. ARM指令的种类，它能完成哪些功能

——共有7类指令，可以完成存储器访问、数据运算、程序跳转、处理器控制、以及帮助编程的伪指令等。

第3章 目录

- 1.ARM处理器寻址方式
- 2.指令集介绍
 - ARM指令集
 - **Thumb指令集**



3.2 指令集介绍

■ Thumb指令

Thumb指令集可以看作是ARM指令压缩形式的子集，它是为减小代码量而提出的，具有16位的代码密度。Thumb指令体系不完整，只支持通用功能。必要时仍需要使用ARM指令，如进入异常时。

■ 简单的Thumb程序

； 功能：使用BX指令切换处理器状态

```
AREA      Example8, CODE, READONLY
```

```
ENTRY
```

```
CODE32
```

```
ARM_CODE
```

```
ADR      R0, THUMB_CODE+1
```

```
BX      R0                      ; 跳转并切换处理器状态
```

```
CODE16
```

在Thumb程序段之前
要用CODE16声明。

```
THUMB_CODE
```

```
MOV      R0, #10                ; R0 = 10
```

```
MOV      R1, #20                ; R1 = 20
```

```
ADD      R0, R1                 ; R0 = R0+R1
```

```
B        .
```

```
END
```

3.2 指令集介绍

■ Thumb指令集与ARM指令集的相似处

Thumb指令的格式与使用方式与ARM指令集类似，它也具有以下种类的指令：

- 存储器访问指令
- 数据处理指令
- 分支指令
- 杂项指令
- 伪指令

3.2 指令集介绍

■ Thumb指令集与ARM指令集的区别

Thumb指令集较ARM指令集有如下限制：

- 只有B指令可以条件执行，其它指令都不能条件执行；
- 分支指令的跳转范围有更多限制；
- 数据处理指令的操作结果必须放入其中一个；
- 单寄存器访问指令，只能操作R0~R7；
- LDM和STM指令可以对R0~R7的任何子集进行操作；

附加小知识

■ C语言编程中声明变量的几个重要关键词

➤ **__packed**

用于将一个结构体在存储器中紧凑存储

➤ **volatile**

用于告诉编译器不要对该变量的操作优化掉

➤ **__align(n)**

用于控制变量的n字节对齐存储

附加小知识

__packed

通常ARM编译器会按以下原则自动存放结构体：

- 起始地址为结构体成员中最大数据宽度的整数倍；
- 通过插入填充位，使对结构体成员的访问可以按对齐访问操作。

```
struct
{
    uint8  X;
    uint32 Y;
    uint8  Z;
    uint16 M;
}tmp;
```

通过指针，无法顺序读取结构体程序

地址					地址
0x4000	X				0x4003
0x4004	Y0	Y1	Y2	Y3	0x4007
0x4008	Z				0x400B
0x400C	M0	M1			0x400F
	00b	01b	10b	11b	

附加小知识

__packed

使用 **__packed** 关键词定义结构体，可以把插入的填充位全部去除，称之为**结构体的紧凑存储**。

__packed struct

```
{  
uint8  X;  
uint32 Y;  
uint8  Z;  
uint16 M;  
}tmp;
```

地址

0x4000
0x4004
0x4008
0x400C

X	Y0	Y1	Y2
Y3	Z	M0	M1

00b 01b 10b 11b

地址

0x4003
0x4007
0x400B
0x400F

附加小知识

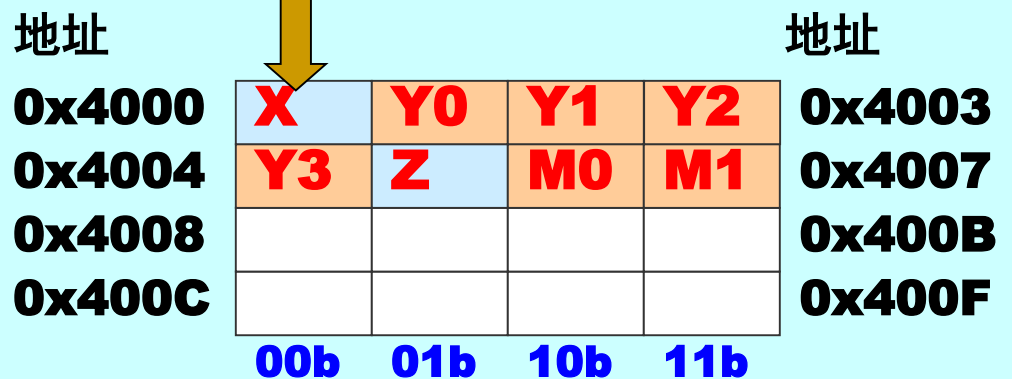
__packed

- 紧凑存储的结构体可以使用指针方便的顺序读取所有成员内容;
- 缩小结构体的存储空间;
- **缺点**: 降低结构体成员的访问速度。

__packed struct

```
{  
uint8  X;  
uint32 Y;  
uint8  Z;  
uint16 M;  
}tmp;
```

通过指针, 可以顺序读取结构体程序



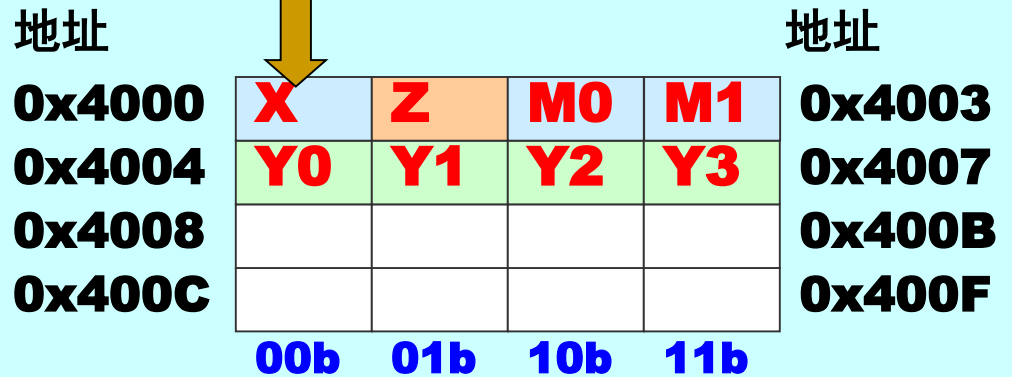
附加小知识

__packed

最理想的编写方式是合理的设计结构体成员的位置，使各成员自然的紧凑存储。

```
struct
{
    uint8  X;
    uint32 Y;
    uint8  Z;
    uint16 M;
}tmp;
```

通过指针，顺序读取结构体程序



附加小知识

volatile

有些变量在中断中被改变，而在正常程序中被读取。这种操作可能被编译器优化，而使得在正常程序中无法获知变量被改变后的内容。

<main.c>

```
uint8 flag;  
...  
while( flag == FALSE )  
{  
    ...  
}  
...
```

编译器认为在一个函数中重复读取一个变量无意义，所以将该操作优化处理。

<ExINT.c>

```
extern uint8 flag;  
void __irq INT(void)  
{  
    ...  
    flag = TRUE;  
    ...  
}
```

附加小知识

volatile

使用**volatile**关键词定义变量后，编译器不会把该变量的任何读写操作优化。

<main.c>

```
volatile uint8 flag;  
...  
while( flag == FALSE )  
{  
    ...  
}  
...
```

<ExINT.c>

```
extern volatile uint8 flag;  
void __irq INT(void)  
{  
    ...  
    flag = TRUE;  
    ...  
}
```

附加小知识

`__align(n)`

变量存放的起始地址和对齐字节数与变量的结构有关，有时为了控制它的对齐方式(比如IAP应用)，可以使用**`__align(n)`**关键词，其中n表示对齐字节数，取值2、4或8。

`uint8 x[8];`

地址

0x4000	?	?	X0	X1
0x4004	X2	X3	X4	X5
0x4008	X6	X7		
0x400C				
	00b	01b	10b	11b

`__align(4) uint8 x[8];`

地址

0x4000	?	?		
0x4004	X0	X1	X2	X3
0x4008	X4	X5	X6	X7
0x400C				
	00b	01b	10b	11b